

Chia dữ liệu và giao thức đánh giá (Data Splitting and Evaluation Protocol)

PHỤ THUỘC B/05-thiet-ke-bai-toan (đơn vị phân tích (*unit of analysis*) quyết định cách chia); B/06-chat-luong-du-lieu (trần đồng thuận và trùng lặp gần đúng). Chương này chuẩn bị trực tiếp cho E/24-thi-nghiem-va-ablation.

TÓM TẮT

Chương này nói về cách để có một con số dự báo được hiệu năng tương lai, và về mọi cách con số đó nói dối. Đọc xong, nhìn một bảng kết quả là bạn biết cách chia của nó giả định gì và metric của nó đang giấu loại lỗi nào. Bạn tính được chênh lệch bao nhiêu điểm mới đáng gọi là thật. Bạn thiết kế được một giao thức đánh giá dám đem ra tranh luận. Nếu chỉ nhớ một điều: mọi lỗi trong chương này đều làm kết quả *đẹp lên* chứ không báo lỗi, nên chúng chỉ bị phát hiện ở chỗ đất nhất — sau khi triển khai — trừ khi bạn đi tìm chúng một cách có hệ thống.

KIẾN THỨC CẦN CÓ TRƯỚC

- **ba tập train / val / test, tham số và siêu tham số (hyperparameter)** — dữ liệu được cắt thành ba phần có ba việc khác nhau: *train* để thuật toán tự tìm ra các tham số của mô hình (trọng số); *val* để bạn chọn các siêu tham số — những số bạn đặt bằng tay trước khi huấn luyện như tốc độ học, số tầng, ngưỡng quyết định — và để biết khi nào nên dừng; *test* chỉ dùng đúng một lần ở cuối để đo. Nhầm vai trò của ba tập này là gốc của mọi vấn đề trong chương.
- **rò rỉ dữ liệu (data leakage)** — bất kỳ đường nào làm thông tin của tập kiểm tra lọt vào quá trình huấn luyện hoặc chọn mô hình — ảnh hưởng nằm hai bên, thống kê chuẩn hoá tính trên cả tập, hai khung hình liền nhau bị tách đôi, hoặc chính bạn xem kết quả test rồi đổi thiết kế. Đặc điểm nguy hiểm là nó không báo lỗi, chỉ làm con số đẹp lên.
- **metric và hàm mất mát (loss function)** — **không phải một** — hàm mất mát là đại lượng *khả vi* mà thuật toán tối ưu giảm xuống trong lúc huấn luyện, tính trên từng mẫu. Metric là đại lượng bạn báo cáo và dùng để ra quyết định, thường không khả vi nên không tối ưu trực tiếp được. Cả chương này nói về metric.
- **accuracy, precision, recall, TPR, FPR và tỉ lệ dương (prevalence)** — accuracy là tỉ lệ đúng trên toàn bộ mẫu. Precision là trong các lần báo “có” thì bao nhiêu phần đúng. Recall, còn gọi là TPR (tỉ lệ dương thật), là trong các ca “có” thật thì bắt được bao nhiêu phần. FPR là trong các ca “không” thật thì bao nhiêu phần bị báo nhầm thành “có”. Prevalence là tỉ lệ ca “có” thật trong toàn bộ dữ liệu — khi nó nhỏ, chẳng hạn 1%, bốn con số trên rời xa nhau tới mức đọc nhầm một cái là kết luận sai hoàn toàn.
- **đường cong ROC, đường cong PR, AUROC và AUPRC** — mọi bộ phân loại đều có một ngưỡng; kéo ngưỡng từ chặt sang lỏng thì được cả một dãy điểm làm việc thay vì một điểm. Vẽ dãy đó trên trục (FPR, TPR) ra đường cong ROC; trên trục (recall, precision) ra đường cong PR. Diện tích dưới mỗi đường là một con số tóm tắt, gọi lần lượt là AUROC và AUPRC.
- **metric riêng của từng bài toán: IoU, mAP, ATE, RPE** — IoU là tỉ số diện tích giao trên diện tích hợp của hai vùng, dùng để quyết định một hộp dự đoán có “khớp” hộp thật hay không; mAP là trung bình diện tích dưới đường PR lấy trên mọi lớp, và “mAP@0.5” nghĩa là coi khớp khi IoU ≥ 0.5 . Trong SLAM, ATE (*absolute trajectory error*) đo độ lệch của toàn bộ quỹ đạo so với quỹ đạo

thật, sau khi căn hai quỹ đạo về cùng hệ toạ độ. RPE (*relative pose error*) đo độ lệch của các đoạn dịch chuyển ngắn.

- **lát (slice), và mẫu (sample) — phân biệt hai nghĩa** — một lát là một tập con của tập kiểm tra được cắt theo một điều kiện có nghĩa vận hành: ảnh ban đêm, vật nhỏ, một camera cụ thể; đọc metric theo từng lát mới thấy được nhóm nào đang bị bỏ rơi. Từ mẫu thì có hai nghĩa: (a) một điểm dữ liệu đơn lẻ, ví dụ một ảnh; (b) một tập được rút ra từ tổng thể, như trong “lấy mẫu có hoàn lại”.
- **sai số chuẩn và khoảng tin cậy (confidence interval)** — sai số chuẩn ước lượng mức dao động của một con số nếu bạn lặp lại phép đo trên một tập khác cùng cỡ; khoảng tin cậy 95% là khoảng rộng khoảng hai lần sai số chuẩn về mỗi phía. Cần biết vì không có nó thì không cách nào phân biệt một cải tiến thật với nhiễu đo đạc.
- **mẫu số của một tỉ lệ** — con số nằm dưới gạch ngang. Nghe tầm thường, nhưng gần như mọi hiểu nhầm về metric là hiểu nhầm mẫu số: precision chia cho số lần hệ thống báo “có”, recall chia cho số ca thật sự dương, còn FPR chia cho số ca thật sự âm. Ba mẫu số khác nhau, nên ba con số kể ba câu chuyện khác nhau về cùng một lỗi.
- **ma trận nhầm lẫn (confusion matrix)** — bảng đếm bốn kết cục có thể xảy ra khi một hệ thống trả lời “có/không” cho một câu hỏi có đáp án “có/không”: báo có và đúng, báo có và sai, báo không và sai, báo không và đúng. Mọi metric phân loại trong chương đều là một phân số lấy từ bốn số này.
- **trung bình điều hoà (harmonic mean)** — nghịch đảo của trung bình cộng các nghịch đảo. Tính chất cần nhớ là nó bị con số nhỏ kéo xuống mạnh hơn trung bình cộng, nên nó thường cho sự cân bằng giữa hai đại lượng. F1 dùng nó chính vì lý do đó.
- **điểm vận hành (operating point)** — một ngưỡng cụ thể mà bạn chọn để biến điểm số liên tục của mô hình thành quyết định “có/không”. Đổi ngưỡng thì mọi metric đổi theo, nên báo cáo một metric mà không nói ngưỡng là báo cáo nửa thông tin.
- **bootstrap** — cách ước lượng độ dao động của một con số khi không có công thức giải tích: lấy mẫu có hoàn lại từ chính tập dữ liệu đã có, tính lại con số, lặp vài nghìn lần, rồi đọc phân vị. Cần nó vì mAP, HOTA hay AUPRC không phải trung bình của các mẫu độc lập nên không có công thức sai số chuẩn.

CÂU HỎI MỞ ĐẦU

- Q1. Vì sao một mô hình đạt 99% accuracy vẫn có thể vô dụng hoàn toàn, và bạn phát hiện điều đó bằng đúng hai con số nào?
- Q2. Bạn không hề chạm vào nhãn của tập test — chỉ tính mean và std trên toàn bộ dữ liệu trước khi chia. Vì sao con số cuối cùng đã hỏng?
- Q3. Vì sao thứ hạng trên leaderboard thường không tái lập được, dù không ai gian lận và không ai làm lộ tập test?
- Q4. Quỹ đạo SLAM của bạn có ATE rất đẹp nhưng chạy thật thì lệch dần theo thời gian. Bước căn chỉnh đã hấp thụ mất cái gì?
- Q5. Mô hình mới hơn mô hình cũ 1 điểm phần trăm trên 1000 ảnh. Phép tính nào cho biết bạn có quyền viết điều đó vào báo cáo hay không?
- Q6. Precision 0,95 nhưng recall 0,40 — hệ thống của bạn đang hành xử thế nào, và ba nguyên nhân nào giải thích được nó?
- Q7. Hai hệ thống có cùng $F_1 = 0,600$. Vì sao một trong hai vẫn có thể tốt hơn hẳn cái kia cho bài toán đóng vòng?
- Q8. Vì sao một mô hình có AUC hoàn hảo vẫn có thể sai hoàn toàn về xác suất, và bạn phát hiện điều đó bằng biểu đồ nào?

1.1 Đặt vấn đề

Bạn cần đúng một thứ từ giao thức đánh giá: *một con số dự báo được hiệu năng tương lai*. Không phải một con số đẹp, cũng không phải một con số so được với paper. Cái bạn cần là: khi bạn nói “mô hình này đạt 0,87”, hệ thống thật sự chạy quanh mức đó trên dữ liệu nó chưa từng thấy.

Điều làm chương này khó chịu là mọi lỗi ở đây đều làm kết quả đẹp lên. **Rò rỉ dữ liệu** (*data leakage*) không báo lỗi, không làm loss tăng, không làm gì cả ngoài việc tặng bạn thêm vài điểm phần trăm và một cảm giác tiến bộ. Không có bug nào có cơ chế tự vệ tốt hơn một bug làm bạn vui.

Người đã tự xây benchmark harness cho SLAM có sẵn phần lớn phản xạ cần thiết, chỉ là chưa gọi tên chúng: bạn biết một lần chạy không nói lên điều gì vì nhiều giữa các lần chạy có thể lớn hơn hiệu ứng cần đo; bạn biết chọn sequence nào để báo cáo là quyết định có sức nặng ngang việc sửa thuật toán; bạn biết một con số ATE trung bình có thể che một lần mất tracking. Chương này là ba trực giác đó viết lại bằng ngôn ngữ học máy, cộng thêm các dạng rò rỉ mà pipeline SLAM không có.

TRỰC GIÁC

Coi tập test như một *mẫu chuẩn niêm phong* trong phòng đo lường, không như một thước đo dùng bao nhiêu lần cũng được. Mẫu chuẩn giữ được giá trị chỉ khi nó ở trong tủ; mỗi lần bạn mở ra để chỉnh thiết bị, một chút đặc tính của thiết bị được in ngược vào cái mà bạn coi là chuẩn. Sau ba mươi lần “chỉnh theo mẫu chuẩn”, thiết bị khớp mẫu chuẩn hoàn hảo và sai với mọi thứ khác. Val là mẫu làm việc, dùng thoải mái và chấp nhận nó hao mòn; test là mẫu niêm phong, mở một lần ở cuối.

1.2 Ba vai trò và các đường rò rỉ (Splits and Leakage)

Việc tách train/val/test hay bị dạy như một thủ tục: cắt 70/15/15. Dạy như vậy thì người học nhớ tỉ lệ và quên mất phần quan trọng hơn — ba tập có *ba vai trò khác nhau về bản chất*.

- **Train** — ước lượng tham số của mô hình.
- **Val** — chọn siêu tham số và quyết định lúc nào dừng.
- **Test** — ước lượng hiệu năng, đúng một lần, ở cuối.

Nhằm vai trò thì mọi kết luận sau đó vô giá trị. Và nhằm vai trò không tạo ra thông báo lỗi nào.

A. Sáu đường rò rỉ, theo thứ tự khó phát hiện dần

Mạch 1 — rò rỉ và cách chặn từng đường

- Vấn đề** đo trên dữ liệu đã học là vô nghĩa.

Giải pháp tách ba tập với ba vai trò rõ ràng.

Giả định bạn thật sự tôn trọng vai trò đó, tức không bao giờ dùng test để quyết định bất cứ điều gì.

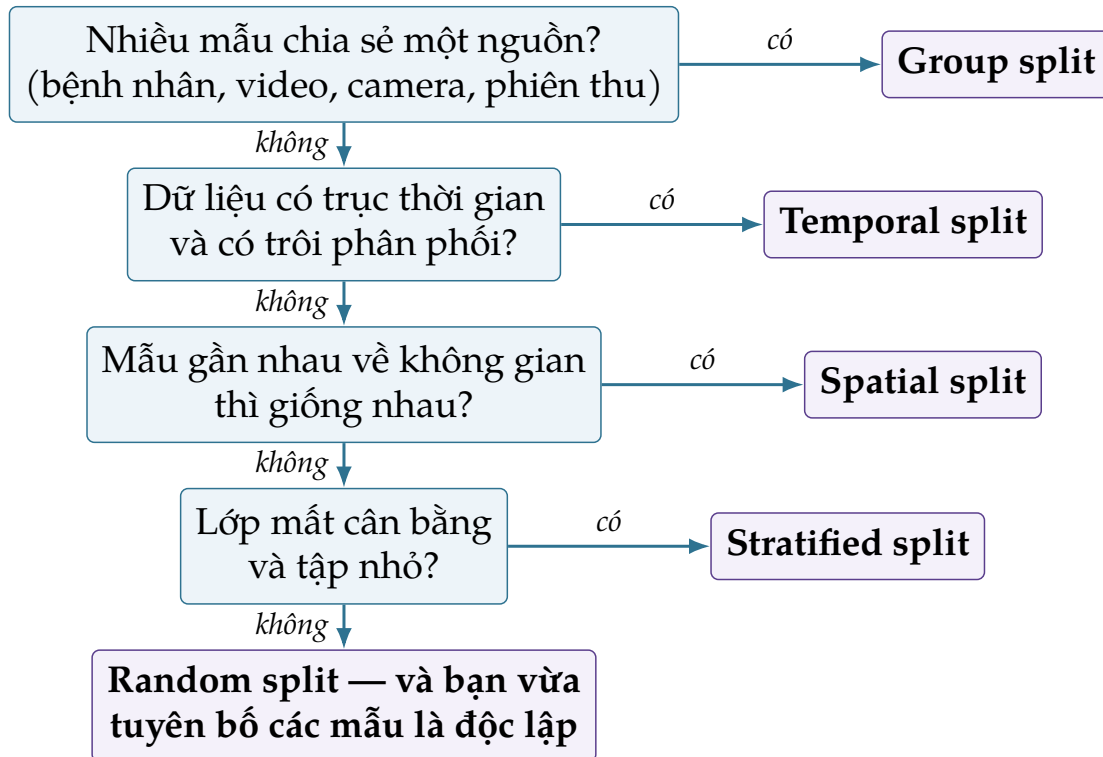
Hệ quả bước duy nhất ai cũng làm, và cũng là bước duy nhất không đủ.
- Vấn đề** rò rỉ qua khâu tiền xử lý.

Giải pháp tính mean/std, PCA, chọn đặc trưng, khớp bộ chuẩn hoá *sau* khi chia và *chỉ* trên train.

- Hệ quả thống kê của test đã đi vào mô hình — nhẹ, nhưng đủ để mọi chênh lệch nhỏ trở nên vô nghĩa.
- 3 Vấn đề** rò rỉ qua mẫu trùng.
- Giải pháp** khử **trùng lặp gần đúng** (*near-duplicate*) trước khi chia, bằng *embedding* chứ không bằng mã băm (B/06).
- Hệ quả với ảnh crawl web và frame video, đây là nguồn tạo kết quả đẹp mà giả phổ biến nhất.
- 4 Vấn đề** rò rỉ qua nhóm.
- Giải pháp** nếu nhiều mẫu chia sẻ một nguồn — cùng bệnh nhân, cùng video, cùng camera, cùng phiên thu — thì mọi mẫu của một nhóm phải nằm trọn một bên.
- Hệ quả bỏ qua điều này là cách rẻ nhất để tự tặng mình mười tới hai mươi điểm hiệu năng ảo; đây là kinh nghiệm được lặp lại rộng rãi trong y tế và video, không phải một con số đo chính xác.
- 5 Vấn đề** rò rỉ qua thời gian.
- Giải pháp** khi dữ liệu có trục thời gian, test phải nằm *sau* train.
- Hệ quả chia ngẫu nhiên trên dữ liệu có trôi cho phép mô hình nội suy giữa quá khứ và tương lai, một đặc quyền nó sẽ không có khi chạy thật.
- 6 Vấn đề** rò rỉ qua chính bạn.
- Vấn đề mới** ***adaptive overfitting***: mỗi lần bạn nhìn test rồi ra một quyết định, một chút thông tin từ test rò vào mô hình qua bộ não bạn.
- Hệ quả sau vài chục vòng như vậy, test đã trở thành val — lời giải thích tự nhiên cho việc thứ hạng leaderboard thường không tái lập được trên một tập test mới thu thập cùng quy trình [1].

B. Chọn cách chia là phát biểu một giả định, không phải chọn một hàm

Nhận thức then chốt của cả chương nằm ở đây. Khi bạn gọi `train_test_split` với `shuffle=True`, bạn không chọn một tiện ích; bạn đang tuyên bố rằng dữ liệu tương lai sẽ đến từ cùng nhóm, cùng thời điểm, cùng địa điểm với dữ liệu hiện tại. Đó là một phát biểu mạnh về thế giới, và nó thường sai. Hình 1.1 biến phát biểu đó thành một chuỗi câu hỏi trả lời được, còn Bảng 1.1 nói cái giá của mỗi lựa chọn.



Hình 1.1. Cây quyết định chọn cách chia. Thứ tự các câu hỏi là thứ tự mức thiệt hại: rò rỉ qua nhóm gây sai lệch lớn nhất nên phải loại trừ trước, còn *stratified* chỉ giảm phương sai chứ không chặn rò rỉ nào — nó đứng cuối vì nó là lựa chọn *cộng thêm*, không phải lựa chọn thay thế. Nhánh cuối cùng không phải mặc định an toàn mà là một tuyên bố cần được biện minh.

Bảng 1.1. Mỗi cách chia là một giả định về cách mô hình sẽ được dùng. Cột cuối nói điều gì xảy ra nếu giả định đó sai.

Cách chia	Giả định tương ứng	Dùng khi	Rủi ro nếu chọn sai
Random	Các mẫu độc lập với nhau	Ảnh rời rạc, không có cấu trúc nhóm	Rò rỉ qua nhóm và qua thời gian, không có dấu hiệu cảnh báo
Stratified	Cần giữ tỉ lệ lớp giữa các tập	Lớp mất cân bằng, tập nhỏ	Giảm phương sai ước lượng nhưng <i>không</i> chặn rò rỉ nhóm
Group	Sẽ gặp nhóm hoàn toàn mới	Y tế, video, đa camera, đa phiên thu	Nếu bỏ qua: hiệu năng ảo cao hàng chục điểm
Temporal	Sẽ dự đoán về tương lai	Có trôi phân phối theo thời gian	Đánh giá lạc quan giả tạo; mô hình được phép nội suy thời gian
Spatial	Sẽ gặp vùng địa lý mới	Ảnh vệ tinh, khảo sát hiện trường	Tự tương quan không gian làm hai bên split gần như trùng nhau

Một hệ quả ít được nói: cách chia phải khớp với đơn vị phân tích đã chọn ở B/05. Chọn frame làm đơn vị rồi chia ngẫu nhiên là công thức tạo rò rỉ. Hai frame cách nhau 33 ms không phải hai quan sát độc lập, đúng như hai keyframe liên kế không cho hai ràng buộc độc lập trong **hiệu chỉnh chùm tia** (*bundle adjustment*). Nếu đơn vị thật sự là “một video” hay “một phiên thu” thì test phải cắt ở cấp đó. Hệ quả là số mẫu độc lập thật nhỏ hơn nhiều so với con số bạn tưởng: một nghìn khung hình lấy từ mười video chỉ đáng giá mười mẫu. Điều đó nói rộng khoảng tin cậy ở cuối chương theo đúng căn bậc hai của tỉ lệ ấy.

↪ **hệ quả của B/05** — đơn vị phân tích chọn ở khâu thiết kế quyết định cách chia nào là hợp lệ; chọn frame rồi chia ngẫu nhiên là rõ ràng ngay từ định nghĩa.

↪ **tiền đề B/06** — khử trùng lặp gần đúng phải xong *trước* khi chia, nếu không thì mọi cách chia đều rõ ràng qua bản sao.

C. Khi dữ liệu quá ít để cắt một tập test tử tế

Kiểm định chéo (*cross-validation*) là câu trả lời cổ điển, và **nested cross-validation** là cách tune siêu tham số mà không làm ô nhiễm ước lượng cuối: vòng trong chọn siêu tham số, vòng ngoài chỉ để đo. Cái giá là chi phí tính toán nhân lên theo số fold. Với deep learning hiện đại, nhân năm lần một lượt huấn luyện kéo dài vài ngày là không khả thi. Đó là lý do *thực dụng*, không phải lý do lý thuyết, khiến kiểm định chéo hiếm gặp trong thị giác máy tính ngày nay. Biết điều này để không nhầm sự vắng mặt của nó với một chuẩn mực đúng.

1.3 Chọn metric khớp chi phí thật (Metric Selection)

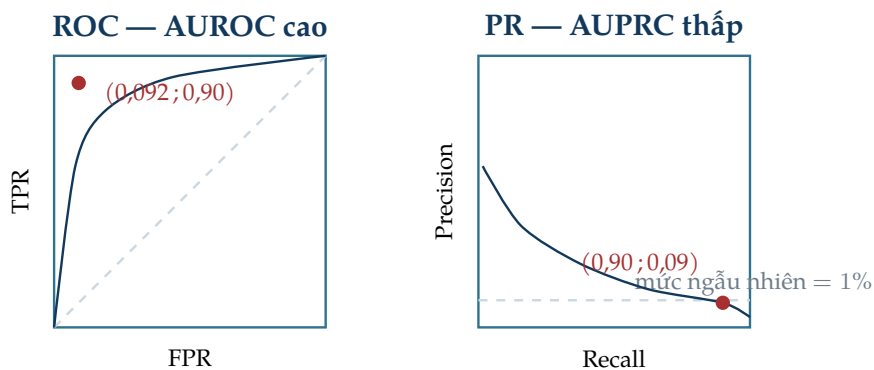
Giả sử đã chia đúng. Vẫn còn một cách nói dối tinh vi hơn: chọn sai thứ để đo. Metric là một hàm nén toàn bộ hành vi hệ thống xuống một số thực, và mọi phép nén đều mất thông tin — câu hỏi duy nhất đáng hỏi là *nó vứt đi loại lỗi nào*.

A. Ví dụ nhỏ nhất: accuracy 99% mà vô dụng

Tập test có 1000 ảnh, trong đó 10 ảnh thuộc lớp dương (1%).

- **Mô hình rỗng** — luôn trả lời “âm”. Accuracy $990/1000 = 99\%$, cao hơn phần lớn hệ thống thật, trong khi hoàn toàn vô dụng.
- **Mô hình có ích** — bắt được 9 trong 10 ca dương (*recall* 0,90) với tỉ lệ báo động giả 9,2% trên lớp âm, tức $0,092 \times 990 \approx 91$ báo động giả.
 - **Trên ROC** — điểm làm việc là $(FPR, TPR) = (0,092, 0,90)$, sát góc trên trái, trông xuất sắc.
 - **Người dùng thấy** — mô hình báo động $9 + 91 = 100$ lần, đúng 9 lần: **precision** = 9%.

Cơ chế rất rõ khi viết ra: FPR chia cho 990 mẫu âm nên một tỉ lệ nhỏ vẫn là rất nhiều mẫu, còn precision chia cho tổng số lần báo động nên nó cảm nhận đúng cái người dùng cảm nhận. Đó là toàn bộ lý do **AUPRC** phải thay **AUROC** khi lớp dương hiếm, và Hình 1.2 cho thấy cùng một điểm làm việc trông khác nhau đến mức nào trên hai không gian.



Hình 1.2. Cùng một mô hình, cùng một điểm làm việc, hai không gian đánh giá (sơ đồ minh họa, không phải số đo thật). Bên trái điểm đó nằm sát góc lý tưởng; bên phải chính nó nằm sát sàn. Khác biệt sinh ra vì trục tung bên trái chuẩn hoá theo 990 mẫu âm, còn bên phải chuẩn hoá theo 100 lần báo động — và người dùng sống ở trục bên phải.

B. Mỗi metric giấu loại lỗi nào

Bảng 1.2. Vài metric quen thuộc và loại lỗi mà mỗi cái làm cho vô hình. Cột giữa là lý do phải đọc ít nhất hai metric cùng lúc.

Metric	Giấu loại lỗi nào	Nên đọc kèm
Accuracy	Toàn bộ hành vi trên lớp hiếm	Precision/recall theo từng lớp
AUROC	Số lượng tuyệt đối báo động giả khi lớp dương hiếm	AUPRC ở đúng mức tỉ lệ dương thật
mAP@0.5	Sai lệch định vị: hộp lệch nhẹ vẫn được tính là đúng	mAP@0.5:0.95, hoặc phân rã lỗi kiểu TIDE [2]
mIoU	Lớp nhỏ và biên; một lớp chiếm phần lớn pixel có thể kéo con số lên	IoU từng lớp, và Panoptic Quality nếu cần tách thực thể [3]
MOTA	Việc liên tục đổi ID: MOTA cao vẫn tương thích với danh tính bị hoán loạn	IDF1 và HOTA [4]
PSNR, SSIM	Chất lượng cảm nhận; ảnh mờ trung bình hoá thường thắng	LPIPS [5], và với mô hình sinh thì FID [6]
ATE sau alignment	Trôi toàn cục, vì phép căn chỉnh hấp thụ đúng thành phần đó	RPE trên nhiều độ dài đoạn [7]

C. Metric hình học và cái bẫy của phép căn chỉnh

Dòng cuối Bảng 1.2 đáng nói kỹ vì người đọc sống ở đó. **sai số quỹ đạo tuyệt đối** (*absolute trajectory error*) được tính *sau* khi căn chỉnh quỹ đạo ước lượng vào quỹ đạo tham chiếu bằng một phép biến đổi cứng, hoặc một phép biến đổi có tỉ lệ trong trường hợp đơn mắt. Phép căn chỉnh này chính là thứ hấp thụ sai lệch toàn cục: một quỹ đạo trôi tỉ lệ đều đặn sẽ bị kéo về gần tham chiếu và cho ra ATE dễ chịu, trong khi lỗi cục bộ vẫn còn nguyên. **sai số tương đối** (*relative pose error*) thì ngược lại — nó đo tính nhất quán trên các đoạn ngắn và mù với sai lệch toàn cục.

Không cái nào sai; sai là đọc một cái rồi kết luận. Nguyên tắc tổng quát: *mỗi metric là một phép chiếu, và bạn cần ít nhất hai phép chiếu không song song để định vị được lỗi*.

→ **cùng nguyên lý với** C/16 — trong thị giác 3D, cặp ATE-RPE và cặp Chamfer-độ phủ đều là hai phép chiếu bù nhau của cùng một sai số.

Một khía cạnh nữa thường bị đẩy ra ngoài giao thức rồi phải trả giá sau là **calibration**: mô hình có thể xếp hạng đúng mà vẫn tự tin sai lệch một cách hệ thống, và mạng hiện đại nổi tiếng là quá tự tin [8]. Nếu đầu ra đi vào một quyết định có ngưỡng, hoặc vào một bộ hợp nhất cảm biến coi nó như xác suất, thì calibration là một phần của tính đúng đắn chứ không phải chuyện phụ — khai triển đầy đủ ở E/25-do-tin-cay-va-van-hanh.

→ **tiền đề** E/25 — một đầu ra xếp hạng đúng nhưng lệch hiệu chuẩn sẽ phá mọi ngưỡng quyết định đặt phía sau nó.

I 1.4 Bốn ô của ma trận nhầm lẫn (Reading the Confusion Matrix)

Từ đây tới hết chương, trọng tâm đổi. Phần trên dạy *chọn* metric; phần này dạy *đọc* metric. Đây là hai kỹ năng khác nhau. Chọn đúng metric mà đọc sai con số thì vẫn ra quyết định sai.

Mọi thứ bắt đầu từ một bảng bốn ô. Một bộ phân loại nhị phân nhận một mẫu và trả lời “có” hoặc “không”. Chân trị cũng chỉ có hai giá trị. Ghép hai bên lại được đúng bốn kết cục, không thừa không thiếu. Mọi metric phân loại mà bạn từng gặp đều là một phân số lắp từ bốn số này.

Người đọc đã sống với bảng này nhiều năm mà chưa gọi tên nó. Trong vòng của SLAM, bước kiểm chứng hình học nhận một cặp khung hình ứng viên và quyết định “đây là cùng một

chỗ” hay “không phải”. Đó chính là một bộ phân loại nhị phân. Ngưỡng số điểm inlier chính là **điểm vận hành** (*operating point*) của nó. Cả chương này vì thế nói về đúng thứ bạn đang chỉnh bằng tay.

A. Bốn ô, gọi bằng lời thường

- **TP — dương thật (*true positive*)**. Hệ thống báo “có” và đúng là có. Đây là công việc hệ thống được thuê để làm.
- **FP — dương giả (*false positive*)**. Hệ thống báo “có” nhưng thật ra không có. Đây là báo động giả. Người dùng nhìn thấy nó ngay.
- **FN — âm giả (*false negative*)**. Hệ thống báo “không” nhưng thật ra có. Đây là ca bị bỏ sót. Người dùng *không* nhìn thấy nó, và đó mới là chỗ nguy hiểm.
- **TN — âm thật (*true negative*)**. Hệ thống báo “không” và đúng là không. Ô này thường đông đảo tới mức vô nghĩa, và chính nó là nguồn của phần lớn ảo giác về hiệu năng.

Ghi nhớ theo một quy tắc: chữ thứ hai là *cái hệ thống nói*, chữ thứ nhất là *hệ thống có nói đúng không*. “False positive” nghĩa là hệ thống nói dương, và nó sai.

Hình 1.3 vẽ bốn ô đó cùng một bộ số cụ thể. Bộ số này đến từ một bộ phân loại tổng hợp mà chúng ta sẽ dùng lại suốt phần còn lại của chương: 20 000 mẫu, trong đó 200 mẫu dương, tức tỉ lệ nền 1%. Ở ngưỡng 0,50, nó cho TP 134, FP 1043, FN 66 và TN 18 757. Đây là số sinh từ mô phỏng, không phải số đo trên một hệ thống thật; giá trị của nó nằm ở chỗ mọi con số phía sau đều tính ra được từ đúng bốn số này.

		CHÂN TRỊ		
		thật sự có (200 ca)	thật sự không (19 800 ca)	
HỆ THỐNG BÁO	báo “có” (1177 lần)	TP = 134 báo có, đúng là có <i>việc hệ thống được thuê để làm</i>	FP = 1043 báo có, thật ra không <i>báo động giả — ai cũng thấy</i>	mẫu số của precision mọi lần hệ thống mở miệng
	báo “không” (18 823 lần)	FN = 66 báo không, thật ra có <i>bỏ sót — không ai thấy</i>	TN = 18 757 báo không, đúng là không <i>ô khổng lồ và gần như vô nghĩa</i>	
		mẫu số của recall mọi ca thật sự có	mẫu số của FPR mọi ca thật sự không	

Hình 1.3. Bốn ô của ma trận nhầm lẫn, với số từ bộ phân loại tổng hợp ở ngưỡng 0,50 (20 000 mẫu, 1% dương; số mô phỏng, không phải số đo). Ba dấu ngoặc là ba mẫu số khác nhau: precision chia theo hàng trên, recall chia theo cột trái, FPR chia theo cột phải. Cùng một ô FP nằm trong hai phân số có mẫu số lệch nhau gần hai mươi lần, và đó là toàn bộ lý do hai metric kể hai câu chuyện khác nhau.

Hai cái bấy nhỏ nhưng tốn thời gian

Trước khi đi tiếp, hai chỗ vấp rất hay xảy ra khi đọc bảng của người khác.

Thứ nhất là *chiều của bảng*. Một số thư viện đặt hàng là lớp thật và cột là lớp dự đoán; một số tài liệu làm ngược lại. Nếu đọc nhầm chiều thì precision và recall hoán vị cho nhau, và bạn sẽ kết luận ngược hoàn toàn. Cách kiểm tra trong ba giây: cộng theo hàng, xem tổng có bằng số mẫu thật của từng lớp không. Nếu có, hàng là chân trị.

Thứ hai là *ai được tính vào mẫu số*. Ở phát hiện vật thể, một dự đoán không khớp hộp nào là FP, nhưng một hộp thật không được khớp là FN — và không có ô TN nào cả, vì số “vùng ảnh không chứa vật” là vô hạn. Đây là lý do accuracy không tồn tại trong phát hiện vật thể, và cũng là lý do mọi metric ở đó đều xây trên precision và recall chứ không xây trên bốn ô đầy đủ.

B. Bốn ô là bốn chi phí, và chính chênh lệch chi phí mới chọn metric

Bốn ô không chỉ là bốn cách đếm. Chúng là bốn loại thiệt hại có giá khác nhau trong đời thực, và cái giá đó do bài toán quyết định chứ không do thuật toán.

- **Đóng vòng trong SLAM.** Một vòng sai (FP) khâu hai chỗ khác nhau lại thành một, làm cong toàn bộ bản đồ, và thường không hồi phục được. Một vòng bị bỏ lỡ (FN) chỉ có nghĩa là sai số tích lũy chưa được xoá lần này; lát nữa qua chỗ đó lần nữa vẫn còn cơ hội. Chi phí lệch hẳn về một phía, có thể tới vài bậc độ lớn — đây là đánh giá định tính từ cơ chế, không phải một con số đo được.
- **Sàng lọc y tế.** Ngược lại hoàn toàn. Một ca bỏ sót (FN) là một người bệnh về nhà. Một báo động giả (FP) chỉ tốn thêm một lần xét nghiệm.
- **Phát hiện vật cản cho robot.** FN là va chạm. FP là robot phanh vô cớ. Cả hai đều tổn, nhưng tổn theo hai đơn vị khác nhau, và người thiết kế phải quy đổi.

Từ đây rút ra nguyên tắc chi phối cả phần còn lại của chương. *Metric nào đáng nhìn phụ thuộc vào ô nào đấy*. Nếu FP đắt thì precision là con số quan trọng. Nếu FN đắt thì recall là con số quan trọng. Nếu cả hai cùng đắt và đắt ngang nhau, bạn cần một con số gộp — và sẽ thấy ở mục sau rằng gộp cũng có giá của nó.

Còn một điều nữa cần nói thẳng. Ô TN gần như không bao giờ đắt. Nó chỉ đông. Trong bảng trên, TN chiếm 93,8% tổng số mẫu. Bất kỳ metric nào có TN trong tử số hoặc trong mẫu số đều bị ô này chi phối, và accuracy là ví dụ tiêu biểu. Đó là lý do accuracy nói dối một cách hệ thống trên dữ liệu lệch lớp.

C. Bắc cầu: kiểm chứng đóng vòng là một bộ phân loại nhị phân

Đặt bảng bốn ô lên đúng hệ thống mà người đọc đang bảo trì. Tầng kiểm chứng hình học của đóng vòng nhận một cặp khung hình ứng viên, chạy RANSAC trên các cặp điểm khớp, đếm số inlier, và so với một ngưỡng. Ánh xạ như sau.

- **Mẫu** là một cặp khung hình ứng viên, không phải một khung hình. Đơn vị phân tích ở đây là cặp, và điều đó ảnh hưởng tới cả cách chia dữ liệu lẫn cách bootstrap về sau.
- **Lớp dương** là “hai khung này thật sự nhìn cùng một chỗ”. Định nghĩa của nó do bạn đặt — thường bằng một ngưỡng khoảng cách trên chân trị quỹ đạo — nên nó là một quyết định thiết kế, không phải một sự thật khách quan.
- **Điểm số** là số inlier sau RANSAC, và **ngưỡng inlier** chính là điểm vận hành.
- **TP** là một vòng đúng được chấp nhận; **FP** là một vòng sai được chấp nhận; **FN** là một vòng đúng bị từ chối; **TN** là một cặp không liên quan bị từ chối, và ô này chiếm gần như toàn bộ dữ liệu.

Đặt xong ánh xạ này thì mọi thứ trong chương trở nên áp dụng được trực tiếp. Nâng ngưỡng inlier từ 20 lên 40 là dịch điểm vận hành sang phải trên đường cong. Việc đó tăng precision, giảm recall, và không đổi một chút nào bản thân bộ mô tả hay bộ ghép cặp. Nếu bạn muốn cải thiện thật sự thì phải làm cả đường cong dịch lên, chứ không phải trượt dọc theo nó — và đó là hai loại công việc hoàn toàn khác nhau.

1.5 Các tỉ lệ dẫn xuất và mẫu số của từng cái (Derived Rates and Denominators)

Gần như mọi hiểu nhầm về metric phân loại là hiểu nhầm mẫu số. Người ta nhớ tử số vì nó là “số ca đúng”, rồi quên hỏi con số đó được chia cho cái gì. Mục này đi qua từng tỉ lệ theo đúng bốn bước: định nghĩa bằng lời, câu hỏi nó trả lời, cao và thấp nghĩa là gì, và điều nó *không* nói.

A. Precision — đọc theo hàng “hệ thống đã báo”

Precision là tỉ lệ đúng trong những lần hệ thống báo “có”. Mẫu số là $TP + FP$, tức toàn bộ số lần hệ thống mở miệng.

$$\text{precision} = \frac{TP}{TP + FP}.$$

Câu hỏi nó trả lời, ở ngôi thứ hai: *trong những cái tôi báo là dương, bao nhiêu phần là đúng?* Nói cách khác, nếu bây giờ hệ thống báo động, bạn có nên tin không.

Precision cao nghĩa là hệ thống ít khi làm phiền bạn vô cớ: mỗi lần nó nói, gần như chắc chắn có chuyện thật. Precision thấp nghĩa là bạn phải tự lọc lại phần lớn những gì nó đưa ra, và niềm tin vào hệ thống bốc hơi rất nhanh.

Điều precision *không* nói: nó không nói gì về số ca bị bỏ sót. Một hệ thống chỉ báo đúng một ca duy nhất trong cả năm, và báo đúng, có precision bằng 1,0. Mẫu số của precision không hề chứa FN, nên FN có thể lớn tùy ý mà precision không nhúc nhích.

B. Recall — đọc theo cột “thật sự có”

Recall, còn gọi là độ nhạy (*sensitivity*) hoặc TPR, là tỉ lệ bắt được trong số các ca thật sự dương. Mẫu số là $TP + FN$, tức toàn bộ số ca dương có trong dữ liệu.

$$\text{recall} = \frac{TP}{TP + FN}.$$

Câu hỏi nó trả lời: *trong những cái thật sự dương, tôi bắt được bao nhiêu phần?*

Recall cao nghĩa là ít thứ lọt lưới. Recall thấp nghĩa là hệ thống im lặng ở phần lớn các ca thật, và sự im lặng đó không để lại dấu vết nào trong log.

Điều recall *không* nói: nó không nói gì về số lần báo thừa. Một hệ thống luôn luôn báo “có” cho mọi mẫu có recall bằng 1,0. Mẫu số của recall không chứa FP.

Hai đoạn trên gộp lại thành một câu đáng thuộc: *precision và recall bỏ qua hai ô khác nhau, nên một mình cái nào cũng vô nghĩa*. Precision mù với FN. Recall mù với FP. Chỉ khi đọc cùng nhau chúng mới khoá được cả bốn ô.

C. Các tỉ lệ còn lại, xếp theo mẫu số

Bảng 1.3 liệt kê các tỉ lệ còn lại. Cột quan trọng nhất là cột mẫu số; cột cuối là cái bẫy đặc trưng của từng cái.

Bảng 1.3. Các tỉ lệ dẫn xuất từ bốn ô. Số ở cột giá trị lấy từ bộ phân loại tổng hợp ở ngưỡng 0,50 (TP 134, FP 1043, FN 66, TN 18 757). Đọc bảng theo cột mẫu số: hai tỉ lệ khác mẫu số thì không so sánh trực tiếp được với nhau.

Tỉ lệ	Mẫu số là gì	Giá trị	Nó không nói điều gì
Precision	mọi lần báo “có”	0,114	Số ca bỏ sót. Precision 1,0 vẫn tương thích với việc bỏ sót 99% ca thật.
Recall = TPR = độ nhạy	mọi ca thật sự dương	0,670	Số lần báo bừa. Luôn báo “có” cho recall 1,0.
Độ đặc hiệu (<i>specificity</i>) = TNR	mọi ca thật sự âm	0,947	Nó gần như luôn đẹp khi lớp âm đông; 0,947 ở đây vẫn ứng với 1043 báo động giả.
FPR = 1 – specificity	mọi ca thật sự âm	0,053	Số lượng báo động giả. 5,3% của 19 800 là 1043 lần.
FNR = 1 – recall	mọi ca thật sự dương	0,330	Cùng thông tin với recall, chỉ đổi hướng; không phải một metric mới.
NPV (giá trị dự đoán âm)	mọi lần báo “không”	0,9965	Nó gần như luôn sát 1 khi lớp dương hiếm, nên hầu như không mang thông tin.
Accuracy	toàn bộ mẫu	0,945	Bị ô TN chi phối. Mô hình rỗng luôn báo “không” đạt 0,990, cao hơn.
Balanced accuracy	trung bình của recall và specificity	0,808	Nó chứa được lệch lớp nhưng vẫn không phản ánh chi phí thật khi FP và FN lệch giá.
Prevalence (tỉ lệ nền)	toàn bộ mẫu	0,010	Không phải metric của mô hình. Nó là tính chất của <i>dữ liệu</i> , và nó quyết định cách đọc mọi dòng trên.

Ba nhận xét đáng rút ra từ bảng.

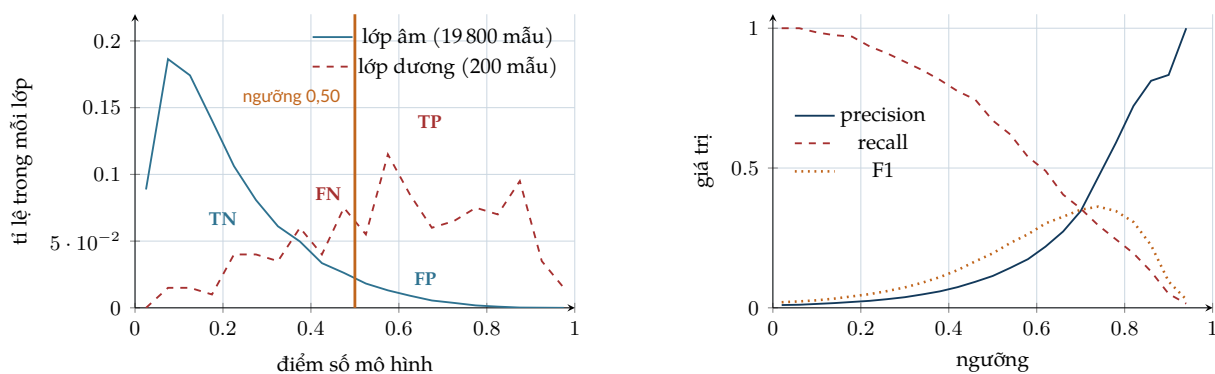
- **Specificity và precision hay bị lẫn.** Cả hai đều “liên quan tới báo động giả”, nhưng specificity chia FP cho số ca âm (19 800), còn precision chia FP cho số lần báo (1177). Mẫu số lệch gần hai mươi lần, nên specificity 0,947 nghe rất đẹp trong khi precision 0,114 thì thảm hại. Cùng một FP, hai câu chuyện.
- **NPV gần như vô dụng khi lớp dương hiếm.** Nếu chỉ 1% mẫu là dương thì đoán “không” đã đúng 99% số lần. NPV cao không chứng minh điều gì.
- **Prevalence không phải metric nhưng phải báo cáo cùng metric.** Precision phụ thuộc trực tiếp vào nó. Cùng một mô hình, đem sang tập có tỉ lệ nền thấp hơn mười lần thì precision tụt gần mười lần, dù mô hình không đổi một chút nào. Recall, specificity và FPR thì không đổi, vì mẫu số của chúng nằm trong một lớp duy nhất.

Nhận xét thứ ba là thứ hay cắn người nhất khi triển khai. Bạn đo precision trên tập test cân bằng nhân tạo, thấy 0,85, rồi đem ra hiện trường nơi tỉ lệ nền thấp hơn hai chục lần và precision rơi xuống 0,2. Không có gì hỏng cả; bạn chỉ đã đo một mẫu số khác.

D. Ngưỡng dịch chuyển bốn ô như thế nào

Bốn ô không phải một tính chất cố định của mô hình. Chúng là một tính chất của mô hình *cộng với* một ngưỡng. Đổi ngưỡng thì bốn số chảy từ ô này sang ô kia theo một quy luật cứng: siết ngưỡng chặt lại thì TP chảy sang FN và FP chảy sang TN; nới lỏng thì ngược lại.

Hình 1.4 cho thấy điều đó trên chính bộ phân loại tổng hợp. Bên trái là phân bố điểm số của hai lớp, bên phải là precision, recall và F1 khi ngưỡng chạy từ lỏng sang chặt. Hai đường precision và recall đi ngược chiều nhau trên toàn dải, và không có ngưỡng nào làm cả hai cùng đẹp — đó là hình dạng của một sự đánh đổi, không phải một khuyết điểm cần sửa.

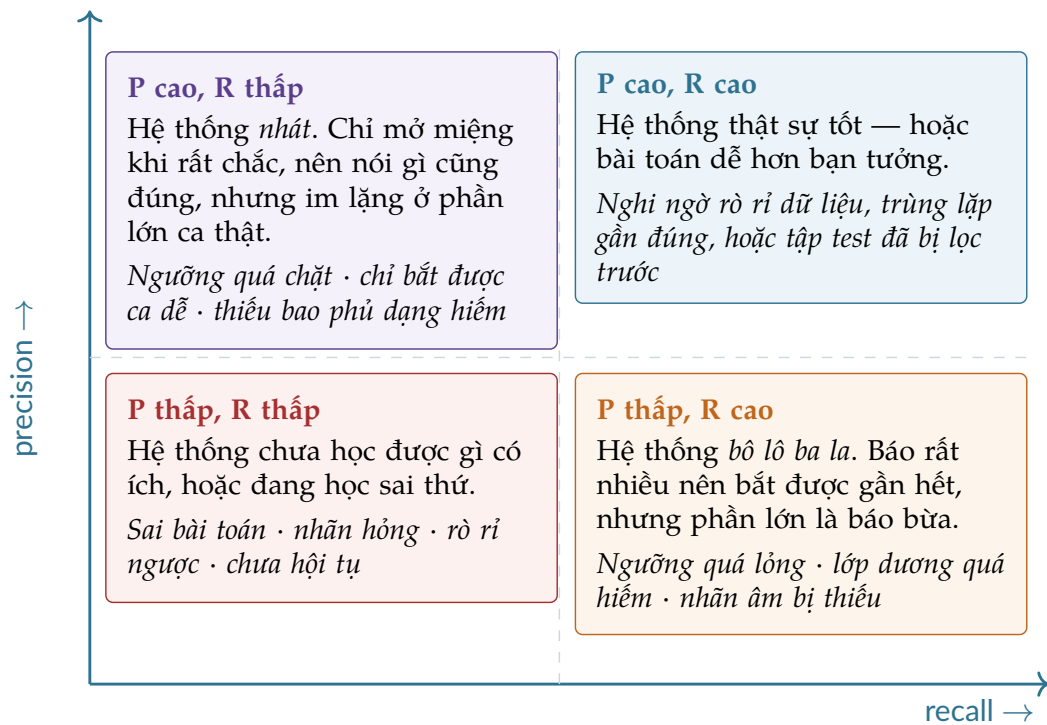


Hình 1.4. Ngưỡng dịch chuyển bốn ô (bộ phân loại tổng hợp, 20 000 mẫu, 1% dương). Bên trái: mỗi đường được chuẩn hoá trong lớp của nó, nên đừng đọc chiều cao như số lượng — đường đỏ đại diện 200 mẫu, đường xanh 19 800 mẫu. TN và FP nằm trên đường xanh (lớp âm); FN và TP nằm trên đường đỏ (lớp dương). Kéo ngưỡng sang phải thì FP giảm và FN tăng. Bên phải: precision và recall đi ngược chiều trên toàn dải, còn F1 đạt cực đại quanh ngưỡng 0,74 — một điểm không có lý do gì để trùng với điểm tối ưu theo chi phí thật của bạn.

Một hệ quả thực dụng: ngưỡng 0,5 không có gì thiêng liêng. Nó chỉ là giá trị mặc định của hàm `argmax` khi có hai lớp. Chọn ngưỡng là một quyết định thiết kế riêng, tách hẳn khỏi việc huấn luyện, và nó phải được ra dựa trên chi phí của FP và FN chứ không dựa trên thói quen.

1.6 Bốn góc precision–recall (The Four Corners)

Đây là phần trả lời trực tiếp câu hỏi “precision cao mà recall thấp thì nghĩa là gì”. Hai con số, mỗi con số cao hoặc thấp, cho bốn tổ hợp. Mỗi tổ hợp mô tả một *hành vi* của hệ thống, và mỗi hành vi lại có vài nguyên nhân khác nhau. Việc của bạn là đi từ tổ hợp về nguyên nhân, và điều đó cần một phép thử chứ không cần trực giác.



Hình 1.5. Bốn góc của mặt phẳng precision–recall và hành vi ứng với mỗi góc. Dòng in nghiêng trong mỗi ô là các nguyên nhân thường gặp — chúng khác nhau về cách sửa, nên phải phân biệt bằng phép thử chứ không đoán. Lưu ý góc trên phải cũng là một cảnh báo: kết quả quá đẹp trên bài toán khó là dấu hiệu của rò rỉ trước khi là dấu hiệu của tài năng.

A. Từ tổ hợp về nguyên nhân, kèm phép thử

Bảng 1.4 là phần dùng được nhất của mục này. Cột cuối là cột đáng giá: nó biến một suy đoán thành một việc làm được trong nửa giờ.

Bảng 1.4. Bốn tổ hợp precision–recall: hệ thống đang hành xử thế nào, vì sao, và làm gì để biết là vì sao. Mỗi ô ở cột giữa liệt kê các nguyên nhân *cạnh tranh nhau*; phép thử ở cột phải là cách tách chúng.

Tổ hợp	Nguyên nhân thường gặp	Phép thử để phân biệt
P cao, R cao	(a) mô hình thật sự tốt; (b) rò rỉ dữ liệu; (c) tập test đã bị lọc bỏ ca khó; (d) bài toán dễ hơn tưởng	So với một baseline ngu ngốc (đoán theo tỉ lệ nền, hoặc một quy tắc thủ công ba dòng). Nếu baseline cũng đẹp thì là (d). Chia lại theo nhóm và đo lại; nếu con số sập thì là (b). Đếm phân bố độ khó của tập test so với dữ liệu hiện trường để bắt (c).
P cao, R thấp	(a) ngưỡng đặt quá chặt; (b) mô hình chỉ học được dạng dễ, mù với dạng hiếm; (c) tập huấn luyện thiếu bao phủ; (d) định nghĩa nhãn dương hẹp hơn thực tế	Quét toàn dải ngưỡng và vẽ đường PR. Nếu nới ngưỡng làm recall tăng mạnh mà precision giảm ít thì là (a) — sửa bằng một dòng cấu hình. Nếu đường PR phẳng lì và tụt ngay thì không phải ngưỡng, mà là (b) hoặc (c). Phân biệt (b) với (c) bằng cách đọc recall theo từng lát.
P thấp, R cao	(a) ngưỡng quá lỏng; (b) lớp dương quá hiếm nên precision bị mẫu số chìm; (c) nhãn âm bị thiếu, cái mô hình bắt đúng lại bị tính là FP; (d) mô hình bám vào một đặc trưng tất	Siết ngưỡng và xem precision có leo lên không — nếu có thì là (a). Nếu precision vẫn thấp ở mọi ngưỡng, tính lại precision kỳ vọng của một bộ đoán ngẫu nhiên (bằng đúng tỉ lệ nền) để xem có phải (b). Với (c), bốc tay 50 mẫu FP và tự gán nhãn lại; tỉ lệ “FP thật ra đúng” nói ngay có phải nhãn thiếu hay không.
P thấp, R thấp	(a) mô hình chưa hội tụ hoặc có bug; (b) nhãn hỏng hoặc mâu thuẫn; (c) đặc trưng đầu vào không mang thông tin về nhãn; (d) rò rỉ ngược — tập huấn luyện chứa thứ mâu thuẫn với tập test	Kiểm tra mô hình có <i>học thuộc</i> nổi một batch nhỏ không; nếu không thì là (a). Đo mức đồng thuận giữa người gán nhãn trên một mẫu con để bắt (b), xem B/06-chat-luong-du-lieu. Nếu mô hình học thuộc được train mà chết trên test, và chia lại vẫn thế, thì nghi (c) hoặc (d).

B. Ba tình huống hai con số này đánh lừa bạn

Bốn góc ở trên giả định precision và recall được tính đúng. Có ba cách rất phổ biến để chúng được tính *sai* mà không ai nhận ra.

- **Accuracy 99% trên dữ liệu lệch 99:1.** Đây là bẫy kinh điển và nó vẫn bắt được người, vì con số 99 nghe như một lời khen. Kiểm tra chỉ mất một phút: tính accuracy của mô hình rỗng luôn trả lời lớp đa số. Nếu con số của bạn không cao hơn nó một cách rõ ràng thì bạn chưa có gì cả. Trong ví dụ đang chạy, mô hình rỗng đạt 0,990 còn mô hình thật đạt 0,945 — *thấp hơn*, dù mô hình thật rõ ràng có ích.
- **Precision đẹp vì tập test đã bị lọc.** Rất hay xảy ra khi tập đánh giá được xây từ đầu ra của một hệ thống cũ: bạn chỉ gán nhãn những ứng viên mà hệ thống cũ đề xuất. Khi đó mọi ca mà hệ thống cũ bỏ sót đều biến mất khỏi mẫu số của recall, và recall bạn đo là recall *có điều kiện đã lọt qua tầng trước*. Nó luôn đẹp hơn recall đầu-cuối, đôi khi đẹp hơn rất nhiều. Dấu hiệu nhận biết: bạn không thể trả lời câu hỏi “các mẫu âm trong tập test đến từ đâu”.
- **Recall đẹp vì chân trị thiếu.** Nếu chân trị bỏ sót một số ca dương thật, thì mẫu số của recall nhỏ hơn thực tế và recall bị thổi lên. Đồng thời, những ca mô hình bắt đúng nhưng chân trị không ghi lại bị tính thành FP, nên precision bị chìm xuống một cách oan uổng. Đây là lý do một mô hình mạnh đôi khi trông tệ hơn mô hình yếu: nó tìm ra những thứ mà người gán nhãn đã bỏ qua. Phép thử duy nhất đáng tin là bốc tay vài chục FP và gán nhãn lại chúng.

Trường hợp thứ ba có một biểu hiện quen thuộc trong SLAM. Khi bạn đánh giá tầng lấy ứng viên đóng vòng bằng nhãn “cùng chỗ” sinh tự động từ chân trị quỹ đạo, ngưỡng khoảng cách bạn chọn *chính là* định nghĩa của lớp dương. Nới ngưỡng từ 1 m lên 3 m thì recall của cùng một hệ thống thay đổi, dù hệ thống không đổi. Con số không mô tả hệ thống; nó mô tả cặp hệ thống–định nghĩa.

1.7 Gộp hai số làm một, và cái giá của việc gộp (F1, F-beta, MCC)

Hai con số khó xếp hạng. Khi phải chọn giữa mười cấu hình, người ta muốn một con số. Mọi cách gộp đều làm mất thông tin, nên câu hỏi đúng vẫn là câu hỏi cũ: nó vứt đi cái gì.

A. Vì sao F1 dùng trung bình điều hoà chứ không phải trung bình cộng

F1 là **trung bình điều hoà** (*harmonic mean*) của precision và recall:

$$F_1 = \frac{2PR}{P+R} = \left(\frac{P^{-1} + R^{-1}}{2} \right)^{-1}.$$

Lý do chọn trung bình điều hoà rất cụ thể: nó bị con số nhỏ kéo xuống. Trung bình cộng thì không. Lấy một hệ thống có precision 0,95 và recall 0,40. Trung bình cộng cho 0,675, nghe khá ổn. F1 cho 0,563, và con số đó phản ánh đúng hơn cái mà người dùng gặp: một hệ thống bỏ sót 60% ca thật.

Nói chặt hơn, trung bình điều hoà luôn nhỏ hơn hoặc bằng trung bình cộng, và khoảng cách giữa hai cái càng lớn khi hai số càng lệch nhau. Bằng nhau chỉ khi $P = R$. Vì vậy F1 thưởng cho sự cân bằng và phạt sự lệch — đúng thứ ta muốn khi *không* có lý do ưu tiên một phía. F1 xuất phát từ thước đo F trong truy hồi thông tin [9].

B. Cái F1 che mất, và F-beta chữa được tới đâu

Ba đoạn số sau nói hết vấn đề của F1. Xét hai hệ thống:

- Hệ A: $P = 0,60$, $R = 0,60$.
- Hệ B: $P = 0,90$, $R = 0,45$.

Cả hai đều có $F_1 = 0,600$. Nhưng đây là hai hệ thống hoàn toàn khác nhau. Hệ B đúng chín trên mười lần nó lên tiếng, đổi lại bỏ sót 55% ca thật. Hệ A sai bốn trên mười lần lên tiếng, nhưng bắt được nhiều hơn hẳn. Nếu bạn đang làm đóng vòng, hệ B tốt hơn hẳn; nếu bạn đang sàng lọc y tế, hệ A tốt hơn hẳn. F1 nói chúng bằng nhau. Đó là ý nghĩa của câu “F1 che mất bất đối xứng chi phí”.

F-beta cho phép nói ra sự bất đối xứng đó:

$$F_\beta = \frac{(1 + \beta^2)PR}{\beta^2 P + R}.$$

Cách nhớ ý nghĩa của β : β là số lần recall quan trọng hơn precision. $\beta = 2$ nghĩa là bạn coi một ca bỏ sót đắt gấp đôi một báo động giả; $\beta = 0,5$ nghĩa là ngược lại. Áp vào hai hệ trên: F_2 cho A là 0,600 và cho B là 0,500, nên nó chọn A. $F_{0,5}$ cho A là 0,600 và cho B là 0,750, nên nó chọn B. Cùng một cặp hệ thống, hai kết luận ngược nhau, và cái quyết định là β chứ không phải dữ liệu.

Bài học không phải “hãy dùng F-beta”. Bài học là: chọn β chính là phát biểu tỉ lệ chi phí giữa FP và FN, nên hãy phát biểu nó ra thành lời trước, rồi mới chọn số. Nếu bạn không nói được “một vòng sai đắt bằng bao nhiêu vòng bỏ lỡ” thì bạn cũng chưa đủ dữ kiện để chọn β .

C. Macro, micro, weighted — ba cách gộp cho ra ba kết luận

Với nhiều lớp, còn một tầng gộp nữa: gộp qua các lớp. Ba cách phổ biến, và chúng có thể xếp hạng ngược nhau.

Ví dụ nhỏ nhất, tính tay được. Ba lớp, 1000 mẫu: A có 900, B có 90, C có 10. Mô hình dự đoán như sau. Trong 900 mẫu A, đúng 891, còn 9 bị gọi thành B. Trong 90 mẫu B, đúng 54, còn 36 bị gọi thành A. Trong 10 mẫu C, đúng 1, còn 9 bị gọi thành A.

Viết ra thành bảng thì tính tay được trong hai phút. Bảng 1.5 là ma trận nhầm lẫn của ví dụ đó, kèm precision và recall từng lớp.

Bảng 1.5. Ma trận nhầm lẫn ba lớp của ví dụ tính tay. Hàng là lớp thật, cột là lớp dự đoán. Đọc theo hàng ra recall, đọc theo cột ra precision — và chính hai hướng đọc đó là gốc của chênh lệch giữa macro và micro.

thật	dự đoán			tổng	recall	precision	F1
	A	B	C				
A	891	9	0	900	0,990	0,952	0,971
B	36	54	0	90	0,600	0,857	0,706
C	9	0	1	10	0,100	1,000	0,182
tổng cột	936	63	1	1000			

Tính F1 từng lớp: lớp A được 0,971, lớp B được 0,706, lớp C được 0,182. Ba cách gộp cho ba con số:

- **Micro** — gộp bốn ô của mọi lớp lại rồi mới tính. Ở bài toán một nhãn, micro-F1 bằng đúng accuracy: $946/1000 = 0,946$.
- **Weighted** — trung bình F1 các lớp, trọng số theo số mẫu mỗi lớp: 0,939.
- **Macro** — trung bình cộng F1 các lớp, mỗi lớp một phiếu: 0,619.

Chênh lệch giữa 0,946 và 0,619 không phải sai số. Nó là hai câu hỏi khác nhau. Micro hỏi “một mẫu ngẫu nhiên có được xử lý đúng không”, nên lớp đông quyết định. Macro hỏi “một lớp ngẫu nhiên có được phục vụ tử tế không”, nên lớp C với 10 mẫu có quyền ngang lớp A với 900 mẫu. Weighted nằm giữa nhưng gần micro tới mức thường không thêm thông tin.

Quy tắc chọn rất đơn giản: *nếu bạn quan tâm tới lớp hiếm thì phải báo macro*. Và luôn báo cả hai. Một cặp micro cao, macro thấp là chữ ký của “con số tổng đang che một lớp bị bỏ rơi”, đúng chủ đề của mục kế tiếp trong chương.

D. MCC và kappa — vì sao chúng ổn định hơn accuracy

hệ số tương quan Matthews (*Matthews correlation coefficient*) dùng cả bốn ô trong một công thức đối xứng [10]:

$$MCC = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}}$$

Nó là hệ số tương quan giữa hai biến nhị phân “dự đoán” và “chân trị”, nên nó nằm trong $[-1, 1]$: 1 là hoàn hảo, 0 là không hơn đoán mò, âm là tệ hơn đoán mò. Tính chất quan trọng nhất: MCC chỉ cao khi cả bốn ô đều tốt. Không ô nào bị bỏ ra ngoài công thức, khác hẳn precision, recall và F1 — cả ba đều không đụng tới TN [11].

Điều đó cho MCC một tính chất mà accuracy không có. Mô hình rỗng luôn trả lời “không” đạt accuracy 0,990 nhưng MCC bằng đúng 0, vì nó không tương quan gì với chân trị. Mô hình thật ở ngưỡng 0,50 đạt accuracy 0,945 — thấp hơn — nhưng MCC 0,261. Hai con số accuracy xếp hạng ngược, MCC xếp hạng đúng.

hệ số kappa của Cohen (*Cohen's kappa*) [12] đi theo cùng ý tưởng nhưng phát biểu khác: nó đo mức đồng thuận vượt trên mức đồng thuận ngẫu nhiên tính từ tỉ lệ nền. Trên cùng ví dụ, kappa bằng 0,181. Cả hai đều là cách trả lời câu hỏi “con số này hơn đoán mò bao nhiêu”, và cả hai vì thế miễn nhiễm với trò 99% accuracy.

Cái giá phải trả cũng nên nói thẳng: MCC và kappa khó diễn giải hơn hẳn. Không ai nói được “MCC 0,26 nghĩa là người dùng gặp bao nhiêu báo động giả”. Chúng tốt để xếp hạng và để phát

hiện ảo giác, không tốt để *báo cáo cho người vận hành*. Trong thực tế, cặp precision–recall vẫn là thứ đi vào báo cáo, còn MCC là thứ bạn nhìn để tự kiểm.

1.8 Đường cong, và cái đường cong giấu đi (PR and ROC Curves)

Một điểm vận hành cho bốn ô. Nhưng ngưỡng là thứ bạn tự chọn, nên báo cáo một điểm duy nhất là báo cáo một lựa chọn chứ không phải một mô hình. Đường cong sinh ra để tách hai thứ đó: nó cho thấy *toàn bộ* các điểm vận hành mà mô hình có thể đạt tới.

A. Hai đường cong, trực là gì và một điểm nghĩa là gì

- **Đường ROC** — trục hoành là FPR, trục tung là TPR (tức recall). Một điểm trên đường là một ngưỡng cụ thể, và nó nói: “ở ngưỡng này, tôi bắt được TPR phần các ca dương, và làm phiền FPR phần các ca âm”. Cả hai trục chuẩn hoá *trong lớp*, nên đường ROC không phụ thuộc vào tỉ lệ nền.
- **Đường PR** — trục hoành là recall, trục tung là precision. Một điểm nói: “ở ngưỡng này, tôi bắt được recall phần các ca dương, và trong những gì tôi báo thì precision phần là đúng”. Trục tung chuẩn hoá theo *số lần báo*, nên đường PR phụ thuộc mạnh vào tỉ lệ nền.

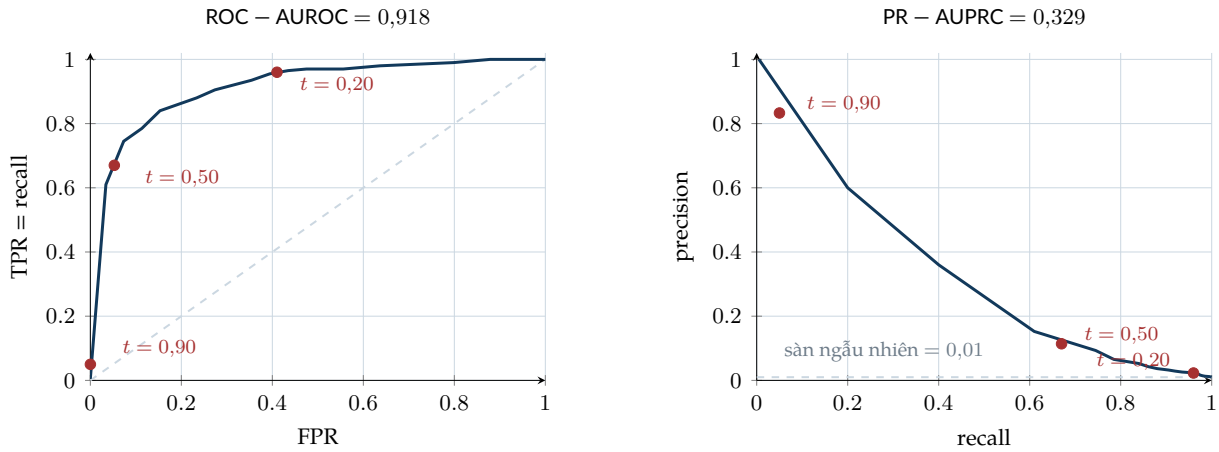
Đường của một bộ đoán ngẫu nhiên nằm ở hai chỗ khác nhau, và đây là chi tiết hay bị quên. Trên ROC, đoán ngẫu nhiên là đường chéo từ (0, 0) tới (1, 1), tức AUROC = 0,5, bất kể tỉ lệ nền. Trên PR, đoán ngẫu nhiên là một đường *nằm ngang ở đúng độ cao bằng tỉ lệ nền*. Với tỉ lệ nền 1%, sàn ngẫu nhiên của đường PR là 0,01. Vì thế AUPRC 0,33 không phải “kém hơn một nửa” như trực giác mách bảo — nó gấp ba mươi ba lần sàn.

B. Vì sao ROC trông đẹp giả tạo khi lớp dương hiếm

Đây là ý quan trọng nhất của cả mục, và nó nằm gọn trong một câu: *FPR có mẫu số rất lớn, nên FP tăng mạnh mà FPR gần như không nhúc nhích*.

Chạy con số cho cụ thể, vẫn trên bộ phân loại tổng hợp. Ở ngưỡng 0,50, FPR bằng 0,053. Nghe nhỏ. Nhưng 0,053 nhân với 19 800 mẫu âm ra 1043 lần báo động giả, trong khi chỉ có 134 lần báo đúng. Người vận hành nhận được 1177 cảnh báo mỗi ngày và 89% trong số đó là rác. Trục ROC không có cách nào hiển thị điều này, vì nó chia cho 19 800 chứ không chia cho 1177.

Hình 1.6 vẽ cả hai đường cho cùng một mô hình. AUROC bằng 0,918 — một con số bạn sẽ vui vẻ viết vào báo cáo. AUPRC bằng 0,329 trên cùng dữ liệu, cùng mô hình, cùng mọi thứ. Không cái nào sai. Chúng trả lời hai câu hỏi khác nhau, và chỉ một trong hai là câu hỏi mà người dùng đang hỏi.



Hình 1.6. Cùng một mô hình, cùng một dữ liệu, hai đường cong (bộ phân loại tổng hợp: 20 000 mẫu, 200 dương). Ba điểm đỏ là cùng ba ngưỡng t trên cả hai hình. Ngưỡng 0,50 nằm gần góc lý tưởng trên ROC nhưng nằm sát sàn trên PR, vì trục tung bên trái chia cho 19 800 mẫu âm còn bên phải chia cho 1177 lần báo động. Sàn ngẫu nhiên của ROC là đường chéo; sàn ngẫu nhiên của PR là đường ngang ở độ cao bằng tỉ lệ nền.

Quan hệ giữa hai không gian đã được chứng minh chặt: một đường cong chi phối trên ROC thì cũng chi phối trên PR và ngược lại, nhưng thứ hạng theo diện tích thì *không* bảo toàn [13]. Trên dữ liệu lệch lớp, đường PR phân biệt các mô hình rõ hơn hẳn [14]. Giới thiệu cổ điển về ROC và cách đọc nó vẫn là bài của Fawcett [15].

C. Chọn điểm vận hành bằng ma trận chi phí, không bằng thói quen

Nếu bạn gán được giá cho FP và FN thì việc chọn ngưỡng không còn là chuyện khẩu vị. Gọi c_{FP} và c_{FN} là chi phí một báo động giả và một ca bỏ sót. Chi phí kỳ vọng ở một ngưỡng là

$$\mathcal{C}(t) = c_{FP} \cdot FP(t) + c_{FN} \cdot FN(t).$$

Quét t qua toàn dải, tính $\mathcal{C}$, lấy điểm nhỏ nhất. Đó là toàn bộ thuật toán. Nó tầm thường về mặt tính toán và gần như không ai làm, vì phần khó là gán giá chứ không phải quét ngưỡng.

Với đóng vòng SLAM, phép tính này ra kết quả rất lệch. Đặt $c_{FN} = 1$ — một vòng bỏ lỡ mất một cơ hội xóa sai số. Đặt $c_{FP} = 100$ — một vòng sai lầm cong bản đồ và thường phải chạy lại. Tỉ lệ 100:1 là một ước lượng bậc thang từ cơ chế, không phải số đo; nhưng ngay cả khi bạn tin nó chỉ đúng trong vòng một bậc, kết luận vẫn không đổi: điểm vận hành phải nằm hẳn ở phần precision cao của đường cong, chấp nhận recall thấp. Đó chính xác là lý do ORB-SLAM3 đòi nhiều khung hình liên tiếp cùng đồng ý trước khi chấp nhận một vòng [16]. Cũng vì thế bộ từ vựng nhị phân luôn đi kèm bước kiểm chứng thời gian và hình học, không ai dùng nó trần [17].

D. Một đường cong đơn điệu vẫn ứng với hai hệ thống rất khác nhau

Đường cong tóm tắt tốt hơn một điểm, nhưng nó vẫn là một phép nén. Hai chỗ nó giấu thông tin:

- **Diện tích bằng nhau, hình dạng khác nhau.** Một mô hình có precision rất cao ở recall thấp rồi tụt dốc, và một mô hình precision trung bình đều trên toàn dải, có thể cho cùng một AUPRC. Nếu bạn chỉ vận hành ở vùng recall thấp thì mô hình thứ nhất tốt hơn nhiều. *Hãy đọc đường cong ở vùng bạn thật sự vận hành, đừng đọc diện tích.*
- **Cùng một đường, hai tập lỗi khác nhau.** Đường cong chỉ đếm số lượng, không nói *mẫu* nào sai. Hai mô hình cùng đường PR có thể sai trên hai nhóm mẫu hoàn toàn tách biệt. Cách phát hiện là đọc metric theo lát, hoặc đơn giản là đếm số mẫu mà hai mô hình bất đồng.

1.9 Xác suất có đáng tin không (Calibration)

Ba mục vừa rồi chỉ dùng *thứ hạng* của điểm số. Đường ROC, đường PR, AUC — tất cả đều không đổi nếu bạn ép điểm số qua một hàm tăng nghiêm ngặt bất kỳ. Điều đó có một hệ quả khó chịu: *một mô hình có AUC hoàn hảo vẫn có thể sai hoàn toàn về xác suất*.

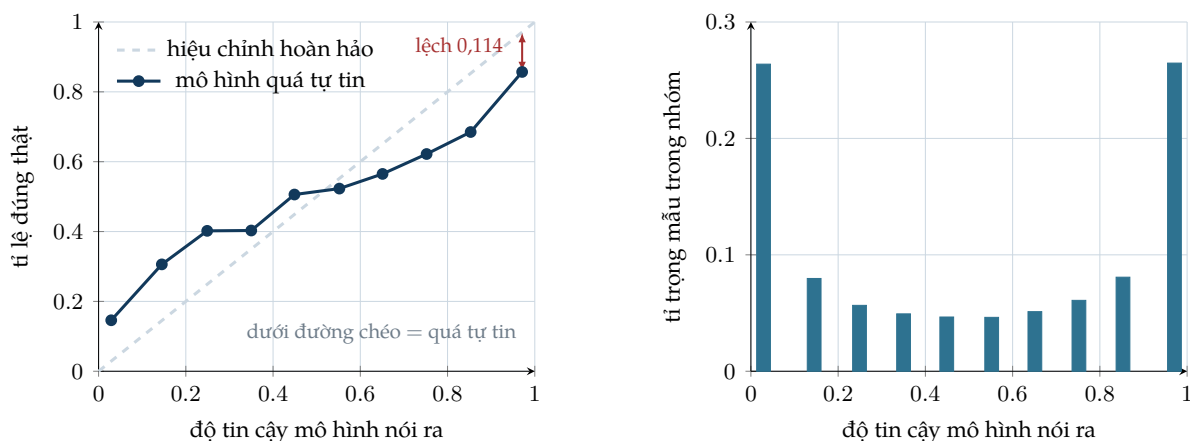
Chuyện này quan trọng khi đầu ra không dừng ở một quyết định nhị phân. Điểm số có thể đi vào một bộ hợp nhất cảm biến, vào một ma trận thông tin, hay vào một quy tắc “chỉ hành động khi độ tin cậy trên 0,9”. Trong cả ba trường hợp, giá trị số của nó là một phần của tính đúng đắn.

A. Biểu đồ độ tin cậy: đọc thế nào

biểu đồ độ tin cậy (*reliability diagram*) được vẽ theo ba bước. Chia các dự đoán thành các nhóm theo độ tin cậy mô hình nói ra — ví dụ mười nhóm, $[0; 0,1)$, $[0,1; 0,2)$, v.v. Trong mỗi nhóm, tính hai số: độ tin cậy trung bình mô hình *nói*, và tỉ lệ đúng *thật*. Vẽ cặp đó lên mặt phẳng. Mô hình hiệu chỉnh tốt nằm trên đường chéo.

Đọc lệch: điểm nằm *dưới* đường chéo nghĩa là mô hình nói mạnh hơn thực tế, tức quá tự tin; nằm trên là quá dè dặt. Mạng nơ-ron hiện đại lệch xuống một cách hệ thống ở vùng độ tin cậy cao [8].

Hình 1.7 vẽ một trường hợp như vậy. Ở nhóm mà mô hình nói 0,97, tỉ lệ đúng thật chỉ 0,86. Điều đáng chú ý nhất: mô hình này có AUROC *y* hết mô hình hiệu chỉnh đúng, vì nó chỉ bị làm sắc lại chứ không đổi thứ hạng. Mọi metric dựa trên thứ hạng đều không thấy gì.



Hình 1.7. Biểu đồ độ tin cậy của một mô hình quá tự tin (dữ liệu tổng hợp, 20 000 mẫu). Bên trái: đường nằm dưới đường chéo ở nửa phải, nghĩa là mô hình nói mạnh hơn thực tế; nhóm “0,97” chỉ đúng 0,86. Bên phải là tỉ trọng mẫu trong từng nhóm, cần thiết vì ECE lấy trung bình có trọng số theo cột này — một nhóm lệch nặng nhưng gần như rỗng thì đóng góp không đáng kể. ECE của mô hình này là 0,115.

B. ECE, Brier, và phân rã Brier

- **ECE** (*expected calibration error*) — lấy độ lệch tuyệt đối giữa hai số trong mỗi nhóm, rồi trung bình có trọng số theo số mẫu trong nhóm [18]. Trong hình trên, ECE bằng 0,115. Đọc là: “trung bình, độ tin cậy mô hình nói ra lệch khoảng 11 điểm phần trăm so với tần suất đúng thật”. Nhược điểm phải biết: ECE phụ thuộc vào cách chia nhóm, nên hai bài báo dùng số nhóm khác nhau không so được với nhau.
- **Điểm Brier** — trung bình bình phương sai lệch giữa xác suất dự đoán và nhãn 0/1 [19]. Nó là một hàm mất mát bình phương, nên nó phạt cả việc xếp hạng sai lẫn việc hiệu chỉnh sai. Ở ví dụ trên, Brier của mô hình là 0,188 còn của mô hình hiệu chỉnh đúng là 0,173.

- **Phân rã Brier** — và đây là chỗ nó trở nên hữu ích thật sự. Điểm Brier tách được thành ba phần: độ tin cậy (*reliability*), độ phân giải (*resolution*) và độ bất định nền (*uncertainty*) [20]. Viết gọn: $\text{Brier} = \text{reliability} - \text{resolution} + \text{uncertainty}$. Phần *reliability* đo đúng cái ECE đo, và càng nhỏ càng tốt. Phần *resolution* đo khả năng mô hình tách các mẫu thành các nhóm có tần suất khác nhau, và càng lớn càng tốt. Phần *uncertainty* chỉ phụ thuộc tỉ lệ nền, không phụ thuộc mô hình.

Phân rã này đáng nhớ vì nó cho một chẩn đoán rõ. Brier tệ do *reliability* lớn thì bạn chỉ cần hiệu chỉnh lại đầu ra, một việc rẻ và hậu kiểm được. Brier tệ do *resolution* nhỏ thì mô hình thật sự không phân biệt nổi các mẫu, và không phép hậu xử lý nào cứu được.

Cách sửa và cách vận hành phần này — *temperature scaling*, hiệu chỉnh theo lát, đo lại sau khi triển khai — thuộc về E/25-do-tin-cay-va-van-hanh. Ở đây chỉ cần nhớ một điều: *AUC cao không cho phép bạn tin con số xác suất*.

1.10 Nhiều lớp và nhiều nhãn (Multi-class and Multi-label)

Bốn ô là chuyện của hai lớp. Với nhiều lớp, ma trận nhầm lẫn thành một bảng $K \times K$, và cách đọc nó là một kỹ năng riêng.

A. Đọc theo hàng và đọc theo cột là hai câu hỏi khác nhau

Quy ước phổ biến: hàng là lớp thật, cột là lớp dự đoán. Khi đó ô (i, j) là số mẫu thuộc lớp i bị gọi thành lớp j . Từ đó:

- **Đọc theo hàng i** — “những mẫu thật sự thuộc lớp i đã đi đâu”. Chia ô đường chéo cho tổng hàng ra *recall của lớp i* . Hàng có ô lớn lệch khỏi đường chéo cho biết lớp i đang bị nuốt bởi lớp nào.
- **Đọc theo cột j** — “những mẫu bị gọi là lớp j thật ra là gì”. Chia ô đường chéo cho tổng cột ra *precision của lớp j* . Cột dày cho biết lớp j là một “hố đen” mà mọi thứ rơi vào.
- **Cặp lớp bị lẫn** — nhìn cặp ô đối xứng (i, j) và (j, i) . Nếu cả hai đều lớn thì hai lớp đó thật sự khó phân biệt, và có thể định nghĩa nhãn đang chồng lẫn. Nếu chỉ một chiều lớn thì đó là thiên lệch một phía, thường do mất cân bằng số lượng.
- **Lớp hiếm biến mất** — một hàng có tổng bằng vài chục trong bảng vài chục nghìn mẫu thì mọi con số tổng đều mù với nó. Đây là lý do phải báo macro cùng micro, như ở mục trước.

B. Top-1 và top-5

Top-1 là accuracy thông thường: nhãn có điểm cao nhất có đúng không. Top-5 hỏi nhẹ hơn: nhãn đúng có nằm trong năm nhãn điểm cao nhất không. Chênh lệch giữa hai con số là một chẩn đoán trực tiếp. Top-5 cao mà top-1 thấp nghĩa là mô hình *biết* vùng đáp án nhưng không xếp hạng nổi trong vùng đó — thường là các lớp cực giống nhau, hoặc nhãn mơ hồ ở chính chân trị. Cả hai cùng thấp nghĩa là mô hình chưa khoanh được vùng.

Cần một cảnh báo: top-5 có ý nghĩa vì ImageNet có 1000 lớp [21]. Với 8 lớp, top-5 gần như luôn bằng 1 và không mang thông tin nào.

C. Nhiều nhãn: subset accuracy so với Hamming

Bài toán nhiều nhãn cho phép một mẫu mang nhiều nhãn cùng lúc. Hai cách đo phổ biến nằm ở hai cực:

- **Subset accuracy** — chỉ tính đúng khi *toàn bộ* tập nhãn khớp chính xác. Rất nghiêm. Với 20 nhãn, một mẫu sai một nhãn cũng bị tính là sai hoàn toàn. Con số này rơi rất nhanh khi số nhãn tăng, và nó phạt như nhau dù bạn sai một nhãn hay sai cả hai mươi.
- **Hamming loss** — tỉ lệ ô nhãn sai trên tổng số ô, tính trên toàn bộ ma trận mẫu $\times$ nhãn. Rất dễ dài. Nếu mỗi mẫu chỉ có 2 trong 20 nhãn là dương, thì đoán “tất cả đều âm” đã cho Hamming loss 0,10, nghe như một kết quả tốt.

Hai cái đó đóng khung khoảng đúng, nên cách hành xử đúng là báo cả hai, cộng với macro-F1 theo nhãn. Riêng Hamming loss cần luôn kèm tỉ lệ nền trung bình, vì nếu không thì nó lặp lại đúng bấy accuracy 99% ở quy mô lớn hơn.

1.11 Metric của từng bài toán thị giác, và điều mỗi cái che giấu (Task-specific Metrics)

Các mục trên là bộ khung chung. Mỗi bài toán thị giác lại đóng gói bộ khung đó theo một cách riêng, và mỗi lần đóng gói lại giấu thêm một thứ. Mục này đi qua năm họ metric mà người đọc sẽ gặp, theo cùng một câu hỏi: *con số này tăng thì thật ra cái gì đã tốt lên*.

A. Phát hiện vật thể: IoU, AP, mAP và các biến thể

Ở phát hiện vật thể, trước khi đếm được TP và FP, phải quyết định một hộp dự đoán có “khớp” một hộp thật hay không. Đó là việc của IoU, tỉ số diện tích giao trên diện tích hợp. Khớp hay không là một quyết định nhị phân với ngưỡng riêng, nên bài toán có *hai* ngưỡng chứ không phải một: ngưỡng độ tin cậy, và ngưỡng IoU.

- **AP** — diện tích dưới đường PR của một lớp, quét theo ngưỡng độ tin cậy với ngưỡng IoU cố định. **mAP** là trung bình AP trên các lớp, tức một phép trung bình macro và vì thế lớp hiếm có quyền ngang lớp đông.
- **AP50 so với AP75** — cùng một mô hình, hai ngưỡng IoU. AP50 dễ dài với sai lệch định vị; AP75 đòi hộp bám sát. Chênh lệch giữa hai con số là một chẩn đoán: AP50 cao mà AP75 thấp nghĩa là mô hình *tìm* được vật nhưng *khoanh* không chuẩn.
- **AP theo kích thước (APs, APm, APl)** — COCO chia vật thể theo diện tích và báo riêng [22]. Đây là cách chia lát đã được đóng sẵn vào metric, và nó là chỗ ẩn nhiều thông tin nhất.
- **AR@k** — recall trung bình khi mỗi ảnh chỉ được giữ k đề xuất. Nó đo chất lượng của tầng sinh ứng viên, tách khỏi tầng phân loại [23]. Cùng tinh thần với Recall@K trong truy hồi ở mục C dưới đây.

Điều mAP che giấu, nói thẳng: “*mAP tăng 1 điểm*” có thể là vật lớn tốt lên trong khi vật nhỏ tệ đi. mAP là trung bình trên các lớp và ngưỡng, nên hai thay đổi ngược chiều có thể triệt tiêu nhau và để lại một con số tổng nhích lên. Nếu ứng dụng của bạn sống bằng vật nhỏ ở xa — đúng trường hợp phát hiện vật cản cho robot — thì con số tổng đang nói dối bạn. Cách chặn rất rẻ: luôn báo APs, APm, APl cùng mAP, và dùng một công cụ phân rã lỗi như TIDE khi cần biết lỗi thuộc loại nào [2], [24].

B. Phân đoạn: mIoU, Dice, boundary F và PQ

- **mIoU** — IoU tính theo pixel cho từng lớp rồi lấy trung bình *theo lớp*. Vì lấy trung bình theo lớp chứ không theo pixel, nó công bằng hơn accuracy pixel; nhưng nó vẫn giấu lớp hiếm theo một cách khác. Một lớp chỉ xuất hiện trong 2% số ảnh vẫn có một phiếu đầy đủ, nên IoU của nó dao động dữ dội giữa các lần chạy và kéo mIoU nhảy theo mà không có nguyên nhân thật.

- **Dice, tức F1 theo pixel** — $\text{Dice} = 2|A \cap B|/(|A| + |B|)$. Nó và IoU là hàm đơn điệu của nhau nên xếp hạng giống nhau, nhưng Dice luôn cho số cao hơn. Đừng so Dice của bài này với IoU của bài kia.
- **Boundary F** — đo riêng độ khớp của *đường biên* thay vì của cả vùng [25]. Cần nó vì IoU bị chi phối bởi phần lõi của vùng: một vật lớn có biên sai vài pixel gần như không làm IoU nhúc nhích, trong khi với một vật nhỏ thì đúng sai lệch ấy là toàn bộ câu chuyện. Đây là điểm đã được nêu từ lâu trong bàn luận về thước đo phân đoạn [26].
- **PQ (*panoptic quality*)** — tách thành hai phần nhân nhau: chất lượng nhận dạng (RQ, chính là F1 ở mức thực thể) và chất lượng phân đoạn (SQ, IoU trung bình của các thực thể đã khớp) [3]. Cấu trúc tích này rất đáng học: nó cho phép nói “PQ tụt vì RQ chứ không vì SQ”, tức vì đếm sai số thực thể chứ không vì khoanh xấu.

C. Truy hồi và nhận dạng địa điểm — và đóng vòng SLAM

Ở truy hồi, hệ thống trả về một danh sách xếp hạng chứ không một câu trả lời. Metric vì thế phải nói rõ “xét bao nhiêu ứng viên”.

- **Recall@K** — tỉ lệ truy vấn mà có ít nhất một kết quả đúng nằm trong K ứng viên đầu. Đây là metric chuẩn của nhận dạng địa điểm. Chú ý mẫu số: nó chia cho *số truy vấn*, không chia cho số ứng viên.
- **Precision@K** — tỉ lệ đúng trong K ứng viên đầu. Nó và Recall@K trả lời hai câu khác nhau, và với đóng vòng thì cái thứ hai mới là cái quan trọng, vì tầng sau phải xử lý cả K ứng viên.
- **mAP truy hồi** — trung bình AP trên các truy vấn, dùng khi một truy vấn có nhiều kết quả đúng.
- **Đường recall-precision theo K** — tăng K thì recall tăng và precision giảm. Đây chính là đường PR của mục trước, chỉ tham số hoá bằng K thay vì bằng ngưỡng.

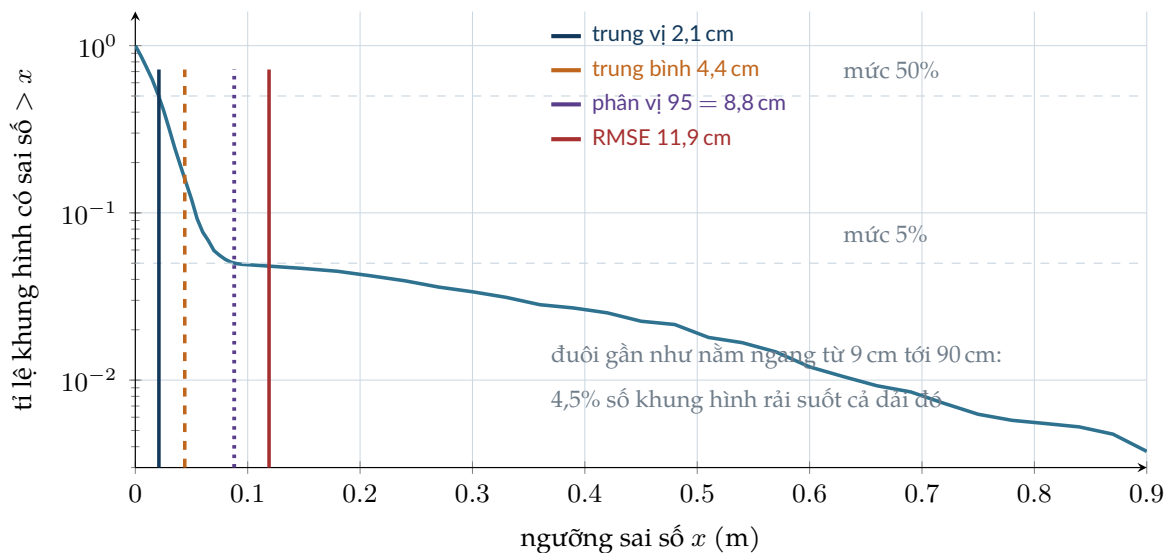
Nói thẳng sang đóng vòng. Tầng lấy ứng viên (DBoW2, NetVLAD, hay bất cứ thứ gì bạn dùng) được đánh giá bằng Recall@K, và điều đó *đúng*, vì việc của nó chỉ là không để lọt. Nhưng hệ thống đóng vòng đầy đủ thì phải được đánh giá bằng precision ở điểm vận hành cuối, sau kiểm chứng hình học. Hai tầng, hai metric, và lẫn lộn chúng là một lỗi thường gặp.

Lý do đã nói ở trên nhưng đáng lặp lại: *một vòng sai đắt hơn nhiều một vòng bỏ lỡ*. Vòng bỏ lỡ chỉ có nghĩa là sai số chưa được xoá; vòng sai khau hai nơi khác nhau thành một và làm hỏng bản đồ, thường không hồi phục. Cho nên điểm vận hành của hệ đóng vòng phải lệch hẳn về phía precision — gần góc trên trái của Hình 1.5 — và recall thấp là một lựa chọn có chủ ý, không phải một thất bại.

D. Hồi quy và quỹ đạo: MAE, RMSE, trung vị, phân vị

- **MAE** — trung bình trị tuyệt đối của sai số. Mỗi mẫu đóng góp tuyến tính.
- **RMSE** — căn của trung bình bình phương sai số. Bình phương làm mẫu lỗi lớn đóng góp gấp bội, nên RMSE là một thước đo *bị đuôi chi phối*.
- **Trung vị và phân vị** — trung vị nói “một mẫu điển hình sai bao nhiêu”; phân vị 95 và 99 nói “ngày tệ nhất tệ tới đâu”.

Hình 1.8 cho thấy vì sao ba nhóm này phải đi cùng nhau. Dữ liệu là sai số vị trí theo từng khung hình của một chuỗi tổng hợp: phần lớn thời gian bám tốt, và 4,5% số khung có một đợt bùng. Trung vị là 2,1 cm, trung bình 4,4 cm, còn RMSE lên tới 11,9 cm. RMSE gấp gần sáu lần trung vị, và toàn bộ chênh lệch đó do 4,5% số khung tạo ra.



Hình 1.8. Cùng một chuỗi tổng hợp (4000 khung), vẽ theo lối “bao nhiêu phần khung hình tệ hơn mức x ” với trục tung log. Đường tụt rất nhanh tới mức 5% rồi *gần như nằm ngang* suốt từ 9 cm tới 90 cm — đó chính là cái đuôi. Bốn đường dọc là bốn cách tóm tắt cùng phân bố này. Trung vị cắt đường ở mức 50%, phân vị 95 cắt ở mức 5%, còn RMSE nằm hẳn bên phải phân vị 95, tức nó mô tả một khung hình bạn gặp ít hơn một lần trong hai mươi. Báo một mình RMSE là báo về cái đuôi và im lặng về hành vi thường ngày; báo một mình trung vị thì ngược lại.

Với quỹ đạo SLAM, hệ quả rất cụ thể và đáng thành thói quen.

- **Báo APE kèm trung vị và phân vị, không chỉ RMSE.** Công cụ [MichaelGrupp/evo](#) in sẵn cả bộ; vấn đề nằm ở việc người ta chỉ chép mỗi dòng RMSE vào bảng. Hai hệ có cùng RMSE có thể là một hệ luôn lệch đều và một hệ hoàn hảo trừ hai giây bung ra. Hướng dẫn đánh giá quỹ đạo của Zhang và Scaramuzza nói rõ điểm này [27]; thư mục metric của TUM RGB-D là nơi cặp APE–RPE được chuẩn hoá [7].
- **Luôn báo tỉ lệ hoàn thành chuỗi cùng APE.** Đây là chỗ dễ tự lừa nhất. Một hệ mất bám ở giây thứ 30 của chuỗi 120 giây, rồi chỉ nộp phần quỹ đạo nó còn giữ, có thể cho APE *rất đẹp* — vì nó chỉ được chấm trên đoạn dễ. So sánh một hệ hoàn thành 100% với một hệ hoàn thành 25% bằng APE là so sánh vô nghĩa. Quy tắc tối thiểu: mọi bảng APE phải có một cột tỉ lệ hoàn thành ngay bên cạnh.

E. Ghép cặp đặc trưng: matching score, MMA, tỉ lệ inlier

- **Matching score** — tỉ lệ điểm đặc trưng phát hiện được mà tìm đúng điểm tương ứng, chuẩn hoá theo số điểm phát hiện được [28].
- **MMA (mean matching accuracy)** — tỉ lệ cặp khớp có sai số chiều dưới một ngưỡng pixel, lấy trung bình trên các cặp ảnh; nó được báo theo từng ngưỡng chứ không một số [29].
- **Tỉ lệ inlier** — tỉ lệ cặp khớp sống sót sau RANSAC. Đây là con số bạn nhìn hằng ngày trong log.

Và đây là câu hỏi mà người đọc thật sự cần trả lời: *vì sao ba con số này tăng mà APE không đổi*. Có bốn lời giải thích, và chúng cần bốn phép thử khác nhau.

- **Bão hoà.** Bộ ước lượng đã có thừa ràng buộc. Thêm cặp khớp đúng thứ 200 không dịch chuyển nghiệm khi 80 cặp đã đủ. Phép thử: giảm số cặp giữ lại xuống một nửa rồi đo lại APE; nếu APE không đổi thì bạn đang ở vùng bão hoà.

- **Đúng nhưng không đa dạng.** Cặp khớp mới tập trung vào vùng đã có nhiều điểm, hoặc ở cùng một mặt phẳng độ sâu. Hình học của bài toán không được cải thiện. Phép thử: đo phân bố không gian của các điểm khớp, và đo số điều kiện của ma trận thông tin trước và sau.
- **Nút thắt nằm ở chỗ khác.** Nếu sai số bị chi phối bởi hiệu chỉnh, độ trễ thời gian, hay mô hình IMU, thì cải thiện tầng khớp không đụng tới nút thắt. Phép thử: thay tầng khớp bằng chân trị (dùng chính chân trị để sinh cặp khớp hoàn hảo) và xem APE tụt tới đâu. Con số đó là trần lý thuyết của mọi cải tiến ở tầng khớp.
- **Metric ghép cặp đo trên phân bố khác.** Matching score và MMA thường đo trên các cặp ảnh có góc nhìn cách xa nhau, còn tracking khung-tới-khung thì góc nhìn gần như nhau. Cải thiện ở chế độ khó không chuyển thành cải thiện ở chế độ dễ. Phép thử: đo lại MMA đúng trên phân bố cặp khung hình mà hệ thống thật sự dùng.

Bài học tổng quát vượt ra ngoài SLAM: *một metric trung gian chỉ có giá trị khi nó nằm trên đường tới hạn của metric cuối.* Trước khi tối ưu một metric trung gian, hãy đo xem nó có nằm trên đường tới hạn không — bằng đúng phép thử “thay bằng chân trị” ở trên.

F. Bảng tóm tắt: mỗi metric trả lời câu hỏi nào

Bảng 1.6 gom năm họ trên vào một chỗ, theo đúng bốn bước của cả khối: nó trả lời câu hỏi nào, mẫu số là gì, và nó im lặng về điều gì.

Bảng 1.6. Metric của từng bài toán, đọc theo cùng một khuôn. Cột “im lặng về” là cột đáng đọc trước, vì nó nói cho bạn biết phải đọc thêm metric nào nữa.

Metric	Câu hỏi nó trả lời	Mẫu số	Nó im lặng về
mAP	Trung bình chất lượng xếp hạng phát hiện trên các lớp	mỗi lớp một phiếu	Vật nhỏ so với vật lớn; chất lượng định vị nếu chỉ dùng ngưỡng IoU 0,5
AR@k	Tầng sinh ứng viên có để lọt vật nào không	số vật thật	Chất lượng phân loại và xếp hạng ở tầng sau
mIoU	Mỗi lớp được khoanh khớp tới đâu, tính theo pixel	mỗi lớp một phiếu	Đường biên; và lớp hiếm có IoU dao động mạnh kéo con số nhảy
Boundary F	Đường biên khớp tới đâu	số điểm biên	Phần lõi của vùng; một vùng lệch hẳn vị trí vẫn có thể có biên trơn
PQ	Vừa đếm đúng thực thể vừa khoanh đúng chưa	thực thể	Cần tách RQ và SQ mới biết hồng ở đếm hay ở khoanh
Recall@K	Ứng viên đúng có nằm trong K kết quả đầu không	số truy vấn	Chi phí xử lý K ứng viên đó ở tầng sau; và precision cuối cùng
APE (ATE)	Quỹ đạo lệch bao nhiêu sau khi căn chỉnh	số khung hình được chấm	Trôi toàn cục đã bị căn chỉnh hấp thụ; và phần chuỗi bị mất bám
RPE	Các đoạn dịch chuyển ngắn có nhất quán không	số đoạn	Sai lệch toàn cục
Tỉ lệ inlier	Bao nhiêu cặp khớp sống sót sau RANSAC	số cặp khớp đề xuất	Hình học có được cải thiện không; và nút thắt có nằm ở tầng khớp không

1.12 Bảng chẩn đoán ngược (Reading Numbers Backwards)

Đây là đích đến của cả khối. Mọi mục trên đi từ định nghĩa tới ý nghĩa. Trong công việc thật, bạn đi ngược: nhìn thấy một bộ số lạ, rồi phải tìm ra nó nghĩa là gì. Bảng 1.7 tổ chức việc đó theo ba cột — triệu chứng, các cách giải thích, và phép thử tách chúng ra.

Cách dùng bảng: đọc cột giữa như một danh sách nghi phạm, không phải một câu trả lời. Gần như mọi triệu chứng đều có từ hai tới bốn nguyên nhân khác nhau, và chúng đòi những cách sửa khác hẳn nhau. Cột phải là thứ biến nghi ngờ thành kết luận.

Bảng 1.7. Từ con số ngược về nguyên nhân. Cột phải luôn là một việc làm được trong một buổi, không phải một lời khuyên chung chung.

Bạn thấy	Các cách giải thích	Phép thử để biết là cách nào
Accuracy 0,99 mà recall lớp hiếm bằng 0	Mô hình đã học chiến lược “luôn đoán lớp đa số”; hoặc lớp hiếm quá ít mẫu để học; hoặc hàm mất mát không có trọng số lớp	Tính accuracy của mô hình rỗng. Nếu bằng nhau thì mô hình đúng là rỗng. Sau đó thử cân bằng lại hàm mất mát và xem recall lớp hiếm có nhúc nhích không; nếu vẫn 0 thì vẫn đề là số mẫu chứ không phải trọng số.
AUROC 0,95 mà AUPRC 0,25	Lớp dương hiếm — đây là hành vi <i>bình thường</i> , không phải lỗi; hoặc mô hình xếp hạng tốt ở vùng giữa mà tệ ở đỉnh danh sách	So AUPRC với tỉ lệ nền: nếu AUPRC gấp hàng chục lần tỉ lệ nền thì mô hình thật sự có ích và bạn chỉ đang đọc nhầm thang đo. Vẽ đường PR và nhìn riêng vùng recall thấp để bắt trường hợp thứ hai.
mAP tăng còn mIoU giảm	Cải thiện ở tầng phát hiện thực thể đi kèm suy giảm ở tầng phân loại pixel; hoặc hai metric được tính trên hai tập lớp khác nhau; hoặc thay đổi hậu xử lý làm hộp chặt hơn còn mặt nạ thô hơn	Đọc AP và IoU theo từng lớp rồi so cặp. Nếu chỉ vài lớp đảo chiều thì là thay đổi thật ở các lớp đó. Kiểm tra danh sách lớp của hai phép đánh giá có trùng nhau không — đây là nguyên nhân tầm thường nhưng rất hay gặp.
Val loss giảm mà metric không đổi	Mô hình đang cải thiện độ tin cậy chứ không đổi thứ hạng; hoặc metric có ngưỡng nên thay đổi nhỏ chưa vượt ngưỡng; hoặc metric bão hoà	Vẽ biểu đồ độ tin cậy trước và sau: nếu ECE giảm mà AUC không đổi thì đúng là chỉ hiệu chỉnh đổi. Quét lại toàn dải ngưỡng; nếu đường PR dịch lên mà điểm vận hành cố định thì là chuyện ngưỡng.
Precision khi triển khai thấp hơn hẳn trên tập test	Tỉ lệ nền ngoài đời thấp hơn trong tập test; hoặc phân phối đầu vào đã dịch chuyển; hoặc tập test đã bị lọc trước bởi một hệ thống cũ	Đo tỉ lệ nền thật ở hiện trường và tính lại precision kỳ vọng theo tỉ lệ đó. Nếu con số khớp thì nguyên nhân là mẫu số, không phải mô hình. Nếu vẫn lệch thì sang B/08-dịch-chuyen-phan-phoi.
F1 đứng yên trong khi cả precision lẫn recall đều đổi	Hai thay đổi bù trừ nhau — một hệ thống khác hẳn đang trộn sau một con số không đổi	Vẽ điểm vận hành cũ và mới trên cùng mặt phẳng PR. Rồi tính F_2 và $F_{0,5}$: nếu chúng xếp hạng ngược nhau thì hai hệ thật sự khác, và β đúng theo chi phí của bạn mới là trọng tài.
Recall@1 cao mà đóng vòng vẫn sai	Tầng lấy ứng viên tốt nhưng tầng kiểm chứng hình học yếu; hoặc nhãn “cùng chỗ” sinh tự động quá lỏng; hoặc lỗi nằm ở tối ưu sau khi chấp nhận vòng	Đếm riêng bốn tầng: ứng viên, ghép đặc trưng, kiểm chứng, tối ưu. Đo precision <i>sau</i> kiểm chứng chứ không trước. Siết ngưỡng khoảng cách của nhãn tự động xuống rồi đo lại Recall@1; nếu nó sập thì nhãn đang quá lỏng.
APE đẹp mà chạy thật thấy trôi	Phép căn chỉnh đã hấp thụ trôi toàn cục; hoặc chuỗi không hoàn thành và chỉ đoạn dễ được chấm	Đọc RPE trên nhiều độ dài đoạn. Kiểm tra tỉ lệ hoàn thành chuỗi trước mọi việc khác — đây là kiểm tra rẻ nhất và hay bị bỏ qua nhất.
APE trung vị tốt mà RMSE tệ	Có một hoặc vài đợt bùng ngắn; hoặc vài khung hình bị lệch mốc thời gian	Vẽ sai số theo thời gian. Nếu là vài đỉnh nhọn thì đọc log ở đúng các mốc đó. Nếu là bậc thang thì nghi mốc thời gian hoặc một lần khởi tạo lại.
Tỉ lệ inlier tăng mà APE không đổi	Bảo hoà ràng buộc; hoặc cặp khớp mới không đa dạng về hình học; hoặc nút thất nằm ở hiệu chỉnh, IMU hay độ trễ	Giảm nửa số cặp khớp và đo lại APE. Thay tầng khớp bằng cặp khớp sinh từ chân trị để lấy trần lý thuyết. Nếu trần đó cũng không tốt hơn hiện tại thì tầng khớp không phải nút thắt.
Macro-F1 thấp hơn micro-F1 rất nhiều	Các lớp hiếm đang bị bỏ rơi — đây gần như luôn là nguyên nhân	Xếp F1 theo lớp từ thấp lên và nhìn năm lớp đáy cùng số mẫu của chúng. Nếu chúng đều là lớp ít mẫu thì đã rõ; nếu có lớp nhiều mẫu nằm trong đáy thì đó là chuyện định nghĩa nhãn.
Metric nhảy mạnh giữa các lần chạy cùng cấu hình	Phương sai do huấn luyện (hạt giống, thứ tự dữ liệu, kernel không tắt định); hoặc tập đánh giá quá nhỏ; hoặc metric nhạy ngưỡng	Chạy lặp năm lần và đo độ lệch chuẩn. So nó với sai số chuẩn tính từ cỡ tập test. Nếu độ lệch giữa các lần chạy lớn hơn hẳn thì nguồn là huấn luyện, không phải cỡ mẫu.

Bạn thấy	Các cách giải thích	Phép thử để biết là cách nào (tiếp)
Mô hình mới thắng ở mAP nhưng thua ở APs	Cải thiện chỉ có ở vật lớn; hoặc thay đổi độ phân giải đầu vào; hoặc gán mẫu đối làm vật nhỏ mất mẫu dương	Báo tách APs, APm, API. Nếu chỉ APs giảm thì kiểm tra độ phân giải và quy tắc gán mẫu trước khi kiểm tra bất cứ thứ gì khác.
Top-5 cao mà top-1 thấp	Các lớp cực giống nhau; hoặc nhãn chân trị mơ hồ với ảnh nhiễu vật	Đọc ma trận nhầm lẫn và tìm các cặp ô đối xứng lớn. Bốc tay 30 ảnh sai và tự gán nhãn lại: nếu bạn cũng phân vân thì đó là trần đồng thuận, không phải lỗi mô hình.
Precision cao ở mọi ngưỡng nhưng recall trần thấp	Mô hình mù với một dạng con của lớp dương; hoặc tăng trước đã lọc mất dạng đó	Đọc recall theo lát. Tìm lát nào có recall gần 0. Rồi kiểm tra dạng đó có xuất hiện trong tập huấn luyện với đủ số lượng không.
ECE tốt mà mô hình vẫn ra quyết định tệ	Hiệu chỉnh tốt trên toàn cục nhưng lệch theo từng lát; hoặc vấn đề là độ phân giải chứ không phải độ tin cậy	Tính ECE riêng theo từng lát. Phân rã điểm Brier: nếu phần reliability nhỏ mà resolution cũng nhỏ thì mô hình không tách được mẫu, và hiệu chỉnh không cứu được.
Hai mô hình cùng AUPRC nhưng người dùng thích hẳn một cái	Hai đường cong cùng diện tích, khác hình dạng; hoặc chúng sai trên hai nhóm mẫu khác nhau	So hai đường cong tại đúng điểm vận hành đang dùng, không so diện tích. Đếm số mẫu hai mô hình bất đồng và xem chúng thuộc lát nào.

1.13 Mọi con số phải kèm một khoảng (Every Number Needs an Interval)

Cả khối trên đọc từng con số như thể nó chính xác. Nó không. Mỗi metric trong chương là một ước lượng tính từ một mẫu hữu hạn, nên nó có một khoảng dao động, và khoảng đó thường lớn hơn mức người ta tưởng.

Phần công cụ chung — sai số chuẩn, so sánh bất cặp, phương sai do huấn luyện — nằm ở mục sau của chính chương này, và kỷ luật thí nghiệm đầy đủ nằm ở E/24-thi-nghiem-va-ablation. Ở đây chỉ nói ba điều riêng của việc *đọc* metric.

- **Mẫu số nhỏ thì khoảng rất rộng, và mẫu số của metric theo lớp luôn nhỏ.** Recall của một lớp có 10 mẫu dương chỉ nhận được 11 giá trị rời rạc, cách nhau 0,1. Nói “recall lớp C tăng từ 0,1 lên 0,2” nghĩa là *một mẫu* đổi kết quả. Đó không phải một cải tiến, đó là một mẫu.
- **Metric không phải trung bình thì không có công thức sai số chuẩn.** mAP, HOTA, PQ, AUPRC đều là hàm phi tuyến của toàn bộ tập, nên công thức $\sqrt{p(1-p)/n}$ không áp dụng được. Cách đúng là *bootstrap* [30]: lấy mẫu có hoàn lại từ chính tập test, tính lại metric, lặp vài nghìn lần, rồi đọc phân vị 2,5 và 97,5. Nó tốn vài phút CPU và trả về đúng thứ bạn cần.
- **Bootstrap phải lấy mẫu theo đúng đơn vị phân tích.** Nếu tập test gồm nhiều khung hình từ ít video, thì lấy mẫu theo khung hình sẽ cho khoảng hẹp giả tạo, vì các khung trong cùng video không độc lập. Phải lấy mẫu theo *video*. Đây là cùng một nguyên tắc đã chi phối cách chia dữ liệu ở đầu chương, xuất hiện lại ở tầng đo độ bất định.

Quy tắc chốt lại: *đo sần nhiều trước, nói về cải tiến sau*. Không có sần nhiều thì mọi so sánh trong bảng chẩn đoán ở trên đều có thể đang chẩn đoán một dao động ngẫu nhiên.

1.14 Con số tổng che nhóm bị bỏ rơi (Slice-based and Behavioral Evaluation)

Ngay cả metric đúng, tính trên tập chia đúng, vẫn là một trung bình — và trung bình che các nhóm nhỏ một cách hoàn hảo. Hai công cụ bù lại chỗ đó, và cả hai đều rẻ:

- **slice-based evaluation** — cắt tập test theo các lát có nghĩa vận hành rồi đọc metric trên từng lát.
 - **Lát nào** — ban đêm, vật nhỏ, mức che khuất cao, từng camera, từng địa điểm, từng đợt thu.
 - **Điều kiện tiên quyết** — phải có metadata cho từng mẫu, nên hãy thu thập nó *từ lúc gán nhãn* (B/06); không có metadata thì không có lát nào cắt được.
 - **Giá trị** — theo kinh nghiệm thực hành, đây là bước có tỉ lệ “phát hiện vấn đề trên mỗi giờ công” cao nhất trong toàn bộ quy trình đánh giá. Khi nhóm tệ nhất mới là thứ quan trọng, có cả một dòng phương pháp tối ưu trực tiếp cho nó [31].
- **kiểm thử hành vi (behavioral testing)** — kiểm tra các tính chất mà bạn *biết* hệ thống phải có, theo kiểu unit test.
 - **Bất biến** — xoay ảnh vài độ thì nhãn không được đổi.
 - **Có hướng** — che vật thể đi thì độ tin cậy phải giảm, không được tăng.
 - **Vì sao cần** — loại test này không cho con số so với paper, nhưng nó bắt đúng loại lỗi mà tập test không bắt được, vì tập test chia sẻ mọi thiên lệch với tập train.

1.15 Chênh lệch 0,3% có thật không? (Uncertainty in Reported Numbers)

Đây là chỗ người đọc có lợi thế sẵn: bạn đã biết chạy cùng một cấu hình ba lần cho ba con số khác nhau, và phần lớn “cải tiến” được báo cáo trong một chiều tối nằm gọn trong phổ đó. Cần biến trực giác này thành phép tính, và có hai nguồn phương sai hoàn toàn khác nhau.

→ **tiền đề** E/24 — kỷ luật chạy lặp và báo cáo độ lệch ở đây chính là điều kiện tối thiểu để một bảng ablation có nghĩa.

- **Phương sai do tập test hữu hạn.** Với 1000 ảnh và accuracy 92%, sai số chuẩn là $\sqrt{0,92 \cdot 0,08/1000} \approx 0,0086$, nên **khoảng tin cậy (confidence interval)** 95% rộng khoảng $\pm 1,7$ điểm phần trăm. Công thức này nói: *nếu lấy một tập test khác cùng cỡ từ cùng phân phối, con số sẽ nhảy trong khoảng đó*. Một chênh lệch 0,3 điểm nằm sâu bên trong và không nói lên điều gì.
 - **Nhưng phải trung thực về một điểm tinh tế** — khi hai mô hình đo trên cùng một tập test, phép so sánh là bất cập, và chỉ những mẫu chúng bất đồng mới mang thông tin. Bất đồng ở 40 mẫu, chia 25 nghiêng về A và 15 về B: chênh lệch quan sát được là 1,0 điểm với sai số chuẩn cỡ $\sqrt{40}/1000 \approx 0,63$ điểm — chặt hơn khoảng tin cậy rời rạc ở trên, nhưng vẫn chưa đủ để tuyên bố A thắng.
 - **Bài học kép** — khoảng tin cậy rời rạc là ước lượng bảo thủ cho so sánh bất cập; và ngay cả phép so sánh chặt hơn cũng thường không cứu được một chênh lệch một điểm.
- **Phương sai do quá trình huấn luyện.** Hạt giống ngẫu nhiên, thứ tự dữ liệu, tính không tắt định của kernel GPU — nguồn này không giảm đi khi bạn thu thập thêm ảnh.
 - **Cách xử lý đúng** — chạy lặp, báo cáo trung bình kèm độ lệch, không chọn lần chạy tốt nhất; đúng kỷ luật bạn đã áp dụng cho benchmark harness của mình. Với metric phức tạp như mAP hay HOTA (không có công thức sai số chuẩn giải tích), dùng **bootstrap**: lấy mẫu có hoàn lại từ chính tập test vài nghìn lần rồi đọc phân vị.
 - **Bối cảnh** — việc báo cáo phương sai theo hạt giống từng bị chỉ ra là điểm yếu có hệ thống của cả lĩnh vực [32]. Trong một số nhánh, phần lớn “tiến bộ” biến mất khi **baseline** được tune ngang bằng [33], [34].

1.16 Giới hạn và trường hợp hỏng

Giao thức chặt tới đâu cũng chỉ đo được cái mà tập test đại diện. Nếu tập test lấy từ cùng một khung lấy mẫu với tập train, mọi thiên lệch chung giữa hai bên vẫn vô hình — chia đúng chỉ chặn rõ ràng, không chặn thiên lệch chung. Đây là ranh giới giữa chương này và chương sau: nội dung ở đây giả định phân phối không đổi, còn khi phân phối đổi thì mọi ước lượng ở đây đều lạc quan.

Nguyên tắc “chỉ chạm test một lần” cũng không sống sót nguyên vẹn: một dự án hai năm sẽ chạm test nhiều lần, không tránh được. Cách xử lý trung thực là ghi lại số lần chạm và thay tập test khi nó đã hao mòn — nghĩa là phải giữ sẵn ngân sách thu thập một tập test mới. Cuối cùng, chia theo nhóm và theo thời gian đều làm tập test nhỏ đi và phương sai lớn lên: một cái giá thật, và với dữ liệu ít bạn đôi khi buộc phải chọn giữa ước lượng thiên lệch có phương sai nhỏ và ước lượng không thiên lệch có phương sai lớn.

CẠM BẤY

Cạm bẫy tốn kém nhất không phải chia sai mà là *chọn tập test sau khi đã nhìn kết quả*. Nó xuất hiện dưới những dạng trông vô hại: bỏ vài sequence “bị lỗi thu”, thêm một tập test thứ hai vì tập đầu “không đại diện”, hay chọn checkpoint theo test vì val nhỏ quá. Cả ba đều là cùng một hành vi, và cả ba đều chuyển test thành val mà không ai ghi vào nhật ký.

CẦN NHỚ

Cách chia là một phát biểu về cách mô hình sẽ được dùng, và metric là một phép chiếu làm mất thông tin. Nên hai câu hỏi phải hỏi trước mọi bảng kết quả là: *cách chia này giả định gì*, và *metric này đang giấu loại lỗi nào*. Với một tập test 1000 mẫu, chênh lệch dưới khoảng một điểm phần trăm chưa phải là chênh lệch. [T2]

1.17 Thực hành

Repo và tài nguyên. [cocodataset/cocoapi](#) là định nghĩa tham chiếu của mAP — khi hai bài báo báo cáo mAP khác nhau, thủ phạm thường là cài đặt chứ không phải mô hình; [Lightning-AI/torchmetrics](#); [voxel51/fiftyone](#) (đánh giá theo lát); [dbolya/tide](#) (phân rã lỗi detection); [MichaelGrupp/evo](#) (ATE/RPE); [photosynthesis-team/piq](#) và [richzhang/PerceptualSimilarity](#) (LPIPS); [GaParmar/clean-fid](#). Danh sách chốt tại tháng 9/2026 và sẽ cũ trong 6–12 tháng; riêng nhóm hạ tầng metric (cocoapi, torchmetrics, evo) bền hơn hẳn tên các mô hình dẫn đầu bảng.

Câu hỏi kiểm tra hiểu. mAP@0.5 và mAP@0.5:0.95 khác nhau ở chỗ nào, và loại cải tiến nào chỉ hiện lên ở cái thứ hai? Vì sao AUROC gây hiểu lầm khi lớp dương chiếm 1%, trong khi AUPRC thì không? Bước căn chỉnh trước khi tính ATE hấp thụ mất loại lỗi nào, và metric nào bù lại chỗ đó?

Bài tập phụ. Tự cài mAP từ đầu — matching theo IoU, xây đường cong precision–recall, nội suy — rồi so với [pycocotools](#) trên cùng một file dự đoán. Chênh lệch sẽ xuất hiện, và việc truy ra nó đến từ quy tắc nội suy, từ cách xử lý mẫu iscrowd, hay từ giới hạn số phát hiện mỗi ảnh, dạy nhiều hơn bất kỳ bài đọc nào về metric.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
●●● Rò rỉ dữ liệu	Bất kỳ đường nào làm thông tin của tập test lọt vào huấn luyện hoặc chọn mô hình. Đặc điểm nguy hiểm là nó không báo lỗi, chỉ làm con số đẹp lên.
●●● Vì sao không được chọn checkpoint theo tập test?	Vì như vậy test đã đóng vai trò của val. Train ước lượng tham số, val chọn siêu tham số và quyết định lúc dừng, test chỉ được đo đúng một lần ở cuối.
●●● Vì sao chuẩn hoá trước khi chia đã là rò rỉ?	Vì mean và std tính trên cả tập test đã đi vào mô hình. Rò rỉ không cần tới nhãn — chỉ cần một thông tin nào đó của test tham gia vào huấn luyện.
●●● Chia theo nhóm	Mọi mẫu chung một nguồn — cùng bệnh nhân, cùng video, cùng camera — nằm trọn một bên. Hai khung hình cách nhau 33 ms không phải hai quan sát độc lập, nên bỏ qua nó là cách rẻ nhất để tự tặng mình hàng chục điểm hiệu năng ảo.
●● Chia theo thời gian	Test nằm sau train trên trục thời gian. Cần khi dữ liệu có trôi; chia ngẫu nhiên ở đây cho mô hình đặc quyền nội suy giữa quá khứ và tương lai, thứ nó không có khi chạy thật.
●●● Chia lại tập năm lần đều cho cùng con số, nhưng triển khai thì sập — nguyên nhân?	Chia đúng chỉ chặn rò rỉ, nó không chặn thiên lệch chung. Nếu train và test cùng lấy từ một khung lấy mẫu thì mọi cách chia đều cho cùng một con số lạc quan. Phân biệt bằng vài trăm mẫu thu ở điều kiện triển khai thật rồi đo lại.
●●● Adaptive overfitting	Test hao mòn dần vì mỗi lần bạn nhìn nó rồi ra quyết định là một lần thông tin rò qua bộ não bạn. Sau vài chục vòng, test đã trở thành val.
●● Thứ hạng leaderboard không tái lập trên tập test mới — nguyên nhân?	Adaptive overfitting ở quy mô cộng đồng: hàng nghìn cấu hình được thử trên một tập test cố định. Phân biệt với tiến bộ thật bằng cách thu một tập test mới theo đúng quy trình cũ rồi đo lại.
●● Prevalence	Tỉ lệ mẫu thuộc lớp dương trong dữ liệu. Khi nó nhỏ, accuracy, precision, recall và FPR rời xa nhau tới mức đọc nhầm một con số là kết luận sai hoàn toàn.
●●● AUPRC	Diện tích dưới đường cong precision–recall. Phải dùng thay AUROC khi lớp dương hiếm, vì nó chuẩn hoá theo số lần báo động chứ không theo số mẫu âm.
●●● Vì sao AUROC gây hiểu lầm khi lớp dương chiếm 1%?	Vì FPR chia cho 990 mẫu âm, nên 9% báo động giả trông nhỏ mà thực ra là 91 lần báo nhầm. Precision chia cho tổng 100 lần báo động và cho ra 9%, đúng cái người dùng cảm nhận.
●● Accuracy 99% trên tập có 1% mẫu dương — đọc thêm gì?	Precision và recall ở đúng điểm làm việc. Một mô hình luôn trả lời “âm” cũng đạt 99%, nên chỉ hai con số đó mới tách được mô hình có ích khỏi mô hình rỗng.
●●● Bốn ô TP, FP, FN, TN	TP báo có và đúng; FP báo có nhưng sai, tức báo động giả ai cũng thấy; FN báo không nhưng thật ra có, tức ca bỏ sót không ai thấy; TN báo không và đúng. Chữ thứ hai là cái hệ thống nói, chữ thứ nhất là nó có nói đúng không.
●●● Mẫu số của precision, recall và FPR	Precision chia cho mọi lần báo “có”. Recall chia cho mọi ca thật sự dương. FPR chia cho mọi ca thật sự âm. Ba mẫu số khác nhau là lý do cùng một ô FP cho ba con số nghe khác hẳn nhau.
●●● Precision cao, recall thấp — hệ thống đang làm gì?	Nó nhất: chỉ báo khi rất chắc nên nói gì cũng đúng, nhưng bỏ sót phần lớn ca thật. Nguyên nhân cạnh tranh: ngưỡng quá chặt, chỉ bắt được dạng dễ, hoặc thiếu bao phủ. Quét toàn dải ngưỡng để tách nguyên nhân đầu khỏi hai cái sau.
●●● Precision thấp, recall cao — hệ thống đang làm gì?	Nó báo bừa: bắt được gần hết nhưng phần lớn cảnh báo là rác. Nguyên nhân: ngưỡng quá lỏng, tỉ lệ nền quá thấp, hoặc nhãn âm bị thiếu nên cái bắt đúng lại bị tính là FP. Bóc tay 50 mẫu FP và gắn nhãn lại để tách nguyên nhân cuối.
●● Vì sao F1 dùng trung bình điều hoà?	Vì trung bình điều hoà bị con số nhỏ kéo xuống, còn trung bình cộng thì không. Với $P = 0,95$ và $R = 0,40$, trung bình cộng cho 0,675 còn F1 cho 0,563 — con số sau phản ánh đúng hơn việc bỏ sót 60% ca thật.

Thuật ngữ / câu hỏi	Trả lời (tiếp)
●●● Macro-F1 thấp hơn micro-F1 rất nhiều — nghĩa là gì?	Các lớp hiếm đang bị bỏ rơi. Micro cho mỗi mẫu một phiếu nên lớp đông quyết định; macro cho mỗi lớp một phiếu. Trong ví dụ ba lớp của chương, micro là 0,946 còn macro chỉ 0,619.
●●● Vì sao ROC trông đẹp giả tạo khi lớp dương hiếm?	Vì FPR có mẫu số rất lớn. FPR 0,053 nghe nhỏ, nhưng nhân với 19 800 mẫu âm ra 1043 báo động giả trong khi chỉ có 134 báo đúng. Đường PR chia cho số lần báo nên nó nhìn thấy điều đó.
●● Biểu đồ độ tin cậy	Chia dự đoán theo độ tin cậy thành nhóm, rồi vẽ độ tin cậy trung bình mô hình nói ra so với tỉ lệ đúng thật. Nằm dưới đường chéo là quá tự tin. Một mô hình quá tự tin có AUC y hết mô hình hiệu chỉnh đúng, vì thứ hạng không đổi.
●● Đọc ma trận nhầm lẫn nhiều lớp theo hàng và theo cột	Theo hàng: mẫu thật sự thuộc lớp này đã đi đâu, chia ra recall. Theo cột: mẫu bị gọi là lớp này thật ra là gì, chia ra precision. Cặp ô đối xứng cùng lớn nghĩa là hai lớp thật sự khó phân biệt.
●● mAP@0.5:0.95	mAP lấy trung bình trên nhiều ngưỡng IoU thay vì chỉ 0,5. Nó là chỗ duy nhất mà cải tiến về độ chính xác định vị hiện ra, vì ở ngưỡng 0,5 hộp lệch nhẹ vẫn được tính là đúng.
● HOTA	Metric theo dõi đối tượng cân bằng giữa phát hiện và giữ đúng danh tính. Cần nó vì MOTA cao vẫn hoàn toàn tương thích với việc ID bị hoán loạn liên tục.
●●● ATE	Sai số quỹ đạo tuyệt đối, tính sau khi căn quỹ đạo ước lượng vào quỹ đạo thật. Nó đo độ lệch toàn cục, nên phải đọc kèm RPE để thấy tính nhất quán trên các đoạn ngắn.
●●● Phép căn chỉnh trước khi tính ATE hấp thụ mất gì?	Trôi toàn cục. Một quỹ đạo trôi tỉ lệ đều đặn bị kéo về sát tham chiếu và cho ra ATE dễ chịu, trong khi lỗi cục bộ vẫn còn nguyên và chỉ RPE mới thấy.
●● Calibration	Mức khớp giữa độ tin cậy mô hình nói ra và tần suất nó đúng thật. Mạng hiện đại thường quá tự tin, nên mọi ngưỡng quyết định đặt sau đầu ra đều lệch theo.
●● Slice-based evaluation	Cắt tập test theo nhóm có nghĩa vận hành — ban đêm, vật nhỏ, từng camera — rồi đọc metric trên từng lát. Điều kiện là có metadata cho từng mẫu, nên phải thu thập từ lúc gán nhãn.
● Kiểm thử hành vi	Kiểm tra các tính chất bạn biết chắc phải đúng, theo kiểu unit test: xoay ảnh vài độ thì nhãn không đổi, che vật thì độ tin cậy phải giảm. Nó bắt loại lỗi mà tập test không bắt được.
●●● Vì sao đóng vòng phải lệch hẳn về precision?	Vì một vòng sai khâu hai nơi khác nhau thành một và làm cong bản đồ, thường không hồi phục; còn một vòng bỏ lỡ chỉ là một cơ hội xoá sai số chưa dùng tới. Recall thấp ở đây là lựa chọn có chủ ý.
●●● Vì sao RMSE của quỹ đạo phải đi kèm trung vị và phân vị?	Vì bình phương làm cái đuôi chi phối. Trong ví dụ của chương, trung vị 2,1 cm còn RMSE 11,9 cm, và toàn bộ chênh lệch đo 4,5% số khung tạo ra.
●● Bootstrap cho metric không phải trung bình	Lấy mẫu có hoàn lại từ chính tập test, tính lại mAP hoặc AUPRC, lặp vài nghìn lần, đọc phân vị. Phải lấy mẫu theo đúng đơn vị phân tích — theo video chứ không theo khung hình — nếu không khoảng sẽ hẹp giả tạo.
●●● Hai mô hình chênh 0,3 điểm trên 1000 ảnh — kết luận?	Chưa kết luận được gì. Sai số chuẩn của accuracy 92% trên 1000 mẫu khoảng 0,86 điểm, nên khoảng tin cậy 95% rộng chừng $\pm 1,7$ điểm. So sánh bất cập chặt hơn nhưng thường vẫn không cứu được một điểm.

●●● phải thuộc ●● nên nắm ● biết là có

I Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
cocodataset/cocoapi	Là định nghĩa vận hành của mAP: quy tắc ghép theo IoU, nội suy đường PR, giới hạn số phát hiện mỗi ảnh	Hạ tầng; hai bài báo lệch mAP thường lệch vì cài đặt chứ không vì mô hình
WILDS (p-lambda/wilds)	Chia theo nhóm và theo miền thật — bệnh viện khác, năm khác, vùng khác — thay vì chia ngẫu nhiên	Bằng chứng rằng “chia theo nhóm” đã thành chuẩn thiết kế benchmark
TIDE (dbolya/tide)	Phân rã một con số mAP thành các loại lỗi tách biệt, đúng ý “metric là phép chiếu”	Cùng dòng với công trình phân tích lỗi detection sớm hơn [24]
ImageNet-v2	Thu một tập test mới theo đúng quy trình gốc để đo phần hiệu năng đã mất do tập test cũ hao mòn	Cách duy nhất đã biết để định lượng adaptive overfitting [1]
MichaelGrupp/evo	Chuẩn hoá việc báo cáo ATE cùng RPE và ghi rõ kiểu căn chỉnh đã dùng	Hạ tầng đánh giá SLAM; chính chỗ bẫy căn chỉnh trở nên hữu hình

MINI PROJECT SLAM · Định lượng rò rỉ do chia ngẫu nhiên theo khung hình · 1–2 ngày

Mục tiêu	biến câu “chia ngẫu nhiên trên video là rò rỉ” thành một con số điểm phần trăm đo được trên chính dữ liệu bạn dùng hằng ngày.
Dữ liệu	ba tới bốn chuỗi EuRoC. Nhãn “cùng chỗ / khác chỗ” lấy từ mini project chương B/06-chat-luong-du-lieu, mở rộng tự động bằng chân trị quỹ đạo: hai khung hình cách nhau dưới 1 m và lệch góc dưới 30° thì gán là cùng chỗ.
Các bước	dựng một bộ phân loại cặp thật đơn giản — một ngưỡng trên điểm tương đồng DBoW2, hoặc hồi quy logistic trên khoảng cách cosine giữa hai vectơ mô tả toàn ảnh. Huấn luyện và đo <i>cùng</i> bộ đó ba lần với ba cách chia: ngẫu nhiên theo khung hình, theo đoạn thời gian liền mạch trong cùng chuỗi, và theo chuỗi (giữ trọn một chuỗi ra ngoài). Mỗi cách chạy năm lần với năm hạt giống, ghi cùng một metric (AUPRC, hoặc precision tại recall 0,9).
Xong khi	bạn viết được một câu dạng “chia ngẫu nhiên theo khung hình cho AUPRC cao hơn chia theo chuỗi X điểm, khoảng tin cậy $\pm Y$ ”, trong đó Y tính từ năm lần chạy chứ không đoán.
Nối với	B/06-chat-luong-du-lieu là nguồn nhãn; B/08-dich-chuyen-phan-phoi vì chia theo chuỗi chính là mô phỏng dịch chuyển miền ở quy mô nhỏ; E/24-thi-nghiem-va-ablation cho cách đọc khoảng tin cậy đó.

MINI PROJECT SLAM · Đường PR của tầng lấy ứng viên đóng vòng, và điểm vận hành theo chi phí · 1–2 ngày

Mục tiêu	biến câu “một vòng sai đắt hơn một vòng bỏ lỡ” thành một ngưỡng cụ thể, chọn bằng phép tính chứ bằng cảm giác.
Dữ liệu	ba tới bốn chuỗi EuRoC. Nhãn “cùng chỗ” sinh từ chân trị quỹ đạo: hai khung hình cách nhau dưới 1 m và lệch góc dưới 30°. Ghi lại luôn kết quả với ngưỡng 2 m để đo xem con số nhạy thế nào với chính định nghĩa nhãn.
Các bước	lấy điểm tương đồng của tầng lấy ứng viên mà hệ bạn đang dùng (DBoW2 hoặc tương đương) cho mọi cặp khung hình đủ xa nhau về thời gian. Quét ngưỡng trên điểm đó, ghi TP, FP, FN, TN ở từng ngưỡng, vẽ cả đường PR lẫn đường ROC trên cùng một trang. Sau đó đặt $c_{FN} = 1$ và thử ba giá trị $c_{FP} \in \{10, 100, 1000\}$, rồi tìm ngưỡng cực tiểu hoá $c_{FP} \cdot FP + c_{FN} \cdot FN$ cho từng giá trị. Cuối cùng lập toàn bộ với nhãn ngưỡng 2 m.
Xong khi	bạn có một hình hai bảng cạnh nhau cho thấy ROC gần như không đổi trong khi PR đổi rõ; một bảng ba dòng “chi phí c_{FP} — ngưỡng tối ưu — precision và recall tại đó”; và một câu nói rõ ngưỡng tối ưu dịch bao nhiêu khi định nghĩa nhãn đổi từ 1 m sang 2 m.

Nối với B/06-chat-luong-du-lieu cho chất lượng nhãn tự động; mini project trước của chính chương này cho cách chia dữ liệu khi đo; E/25-do-tin-cay-va-van-hanh cho việc biến điểm tương đồng thành một xác suất đáng tin trước khi đặt ngưỡng.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì với tỉ lệ dương 1%, một mô hình luôn trả lời “âm” đã đạt 99% . Hai con số cần thêm là precision và recall ở đúng điểm làm việc — hoặc tương đương, AUPRC thay cho AUROC.
- A2. Vì thống kê của tập test đã đi vào mô hình qua bộ chuẩn hoá. Rõ ràng không cần nhãn: chỉ cần một thông tin nào đó của test tham gia vào quá trình huấn luyện hay chọn mô hình là con số đã lạc quan.
- A3. Vì adaptive overfitting: cả cộng đồng thử hàng nghìn cấu hình trên một tập test cố định, nên phần thắng cuối cùng có một phần là khớp riêng tập đó. Thu một tập test mới cùng quy trình làm phần đó bốc hơi.
- A4. Hấp thụ trôi toàn cục: phép biến đổi cứng (hoặc có tỉ lệ, với đơn mắt) kéo quỹ đạo trôi đều đặn về sát tham chiếu. RPE bù lại chỗ đó vì nó đo tính nhất quán trên các đoạn ngắn và mù với sai lệch toàn cục.
- A5. Với 1000 ảnh, khoảng tin cậy 95% của một con số 92% rộng khoảng $\pm 1,7$ điểm. So sánh bất cặp chặt hơn: nếu hai mô hình bất đồng ở 40 mẫu chia 25/15 thì sai số chuẩn của chênh lệch cỡ 0,63 điểm — vẫn chưa đủ để tuyên bố thắng.
- A6. Nó đang *nhất*: chỉ mở miệng khi rất chắc, nên nói gì cũng đúng, nhưng im lặng ở 60% ca thật. Ba nghi phạm là ngưỡng quá chặt, mô hình chỉ bắt được dạng dễ, và tập huấn luyện thiếu bao phủ dạng hiếm. Quét toàn dải ngưỡng để tách nghi phạm thứ nhất khỏi hai cái còn lại.
- A7. Vì F1 gộp hai số bằng một phép trung bình đối xứng nên nó che mất bất đối xứng chi phí. Hệ $P = 0,90$, $R = 0,45$ và hệ $P = R = 0,60$ cùng cho $F_1 = 0,600$; với đóng vòng, nơi một vòng sai đắt hơn nhiều một vòng bỏ lỡ, hệ đầu tốt hơn hẳn. $F_{0,5}$ nói ra điều đó, F1 thì không.
- A8. Vì AUC chỉ dùng thứ hạng của điểm số, mà thứ hạng không đổi khi bạn ép điểm số qua một hàm tăng bất kỳ. Biểu đồ độ tin cậy phát hiện chỗ lệch: điểm nằm dưới đường chéo nghĩa là mô hình nói mạnh hơn thực tế. ECE và phân rã điểm Brier định lượng phần lệch đó.

NÓI LẠI THẬT ĐƠN GIẢN

Hãy nghĩ về một kỳ thi. Bạn có ba tập giấy: sách bài tập để học, một đề thi thử để biết mình yếu chỗ nào, và một đề thi thật nghiêm phong để chấm điểm cuối cùng. Điểm của đề thật chỉ có nghĩa khi thí sinh chưa từng nhìn thấy nó.

Bài toán gốc là mọi cách để đề bị lộ mà không ai cố ý làm lộ. Bạn photo đề thật lẫn vào sách bài tập rồi phát nhầm một trang. Bạn ra đề thử bằng đúng các câu của đề thật, chỉ đổi số. Bạn cho lớp này thi trước rồi mới thi lớp sau. Hoặc tình vi nhất: bạn cho thí sinh làm đề thật ba mươi lần, mỗi lần xem điểm rồi chỉnh cách ôn. Sau ba mươi lần, điểm cao không còn nói lên năng lực nữa, nó chỉ nói rằng họ đã quen với đúng tờ đề đó.

Mẹo có ba phần. Một, quyết định trước ai vào tập nào theo đúng cách bạn sẽ gặp người mới: nếu ngoài đời bạn sẽ gặp học sinh trường khác thì đề thật phải là học sinh trường khác. Hai, đừng đọc mỗi con số tổng — xem điểm theo từng nhóm, và luôn hỏi chênh lệch nửa điểm giữa hai thí sinh có lớn hơn sai số của chính kỳ thi hay không.

Ba, và đây là phần dễ quên nhất: mỗi lần thấy một tỉ lệ, hãy hỏi nó chia cho cái gì. “Chín mươi phần trăm” là chín mươi phần trăm của *cái gì*. Một người gác cổng chặn nhầm 5% số khách vào toà nhà nghe như rất giỏi; nhưng nếu mỗi ngày có hai vạn khách và chỉ hai trăm kẻ gian, thì 5% ấy là một nghìn người bị chặn oan, so với hơn một trăm kẻ gian bắt được. Cùng một sai lầm, chia cho hai thứ khác nhau, cho ra hai câu chuyện trái ngược. Con số nào cũng chỉ có nghĩa cùng với mẫu số của nó.

Cái giá là đề thi thật trở thành thứ tiêu hao: dùng ít lần thôi, và phải để dành ngân sách ra một đề mới.

Chỗ không rút gọn được: việc “chênh lệch này có thật không” thì không rút gọn được thành một quy tắc. Đo bằng khoảng dao động của từng con số thì quá dè dặt, mà chỉ đếm những chỗ hai bên trả lời khác nhau thì lại quá dễ dãi khi hai bên vốn rất giống nhau.

Một câu: Mọi kỹ thuật trong chương chỉ bảo vệ một thứ duy nhất: một con số đo trên phần dữ liệu mà bạn chưa từng chạm vào, kể cả bằng ánh mắt.

1.18 Kết nối

Chương này là hệ quả trực tiếp của B/05-thiet-ke-bai-toan: đơn vị phân tích chọn ở đó quyết định cách chia hợp lệ ở đây. Nó dựa trên B/06-chat-luong-du-lieu ở hai chỗ — trùng lặp gần đúng là một đường rò rỉ, và trần đồng thuận quyết định chênh lệch nào đáng bàn. Nó là tiền đề của B/08-dich-chuyen-phan-phoi, chương gỡ bỏ đúng giả định mà toàn bộ chương này đứng lên: rằng test và thực tế cùng một phân phối. Phần calibration nối sang E/25-do-tin-cay-va-van-hanh, còn kỷ luật thí nghiệm và ablation được khai triển đầy đủ ở E/24-thi-nghiem-va-ablation.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao chuẩn hoá dữ liệu trước khi chia là rò rỉ, dù bạn không hề dùng nhãn của test?
2. Trong Hình 1.1, vì sao câu hỏi về nhóm đứng trước câu hỏi về mất cân bằng lớp?
3. Một mô hình đạt accuracy 99% trên tập có 1% mẫu dương. Cần thêm đúng hai con số nào để biết nó có ích hay không?
4. Vì sao ATE và RPE phải đọc cùng nhau, và phép căn chỉnh làm mất thông tin gì?
5. Adaptive overfitting không tạo ra bug, không tạo ra cảnh báo. Vậy dấu hiệu sớm nào cho thấy tập test của bạn đã hao mòn?
6. Hai mô hình chênh nhau 1 điểm phần trăm trên 1000 ảnh. Phép tính nào cho biết chênh lệch đó có ý nghĩa không, và vì sao khoảng tin cậy rời rạc lại là ước lượng bảo thủ ở đây?
7. Chạy lặp với nhiều hạt giống làm giảm nguồn phương sai nào, và *không* làm giảm nguồn nào?
8. Precision 0,95 và recall 0,40: hệ thống đang hành xử thế nào, và phép thử nào tách “ngưỡng quá chặt” khỏi “mô hình mù với dạng hiếm”?
9. Vì sao AUROC gần như không nhúc nhích khi số báo động giả tăng gấp mười lần trên dữ liệu có 1% mẫu dương, và metric nào thì có?

Nguồn

- [1] B. Recht, R. Roelofs, L. Schmidt, et al., “Do ImageNet Classifiers Generalize to ImageNet?” In *ICML*, 2019.
- [2] D. Bolya, S. Foley, J. Hays, et al., “TIDE: A General Toolbox for Identifying Object Detection Errors,” in *ECCV*, 2020.
- [3] A. Kirillov, K. He, R. Girshick, et al., “Panoptic Segmentation,” in *CVPR*, 2019.
- [4] J. Luiten, A. Osep, P. Dendorfer, et al., “HOTA: A Higher Order Metric for Evaluating Multi-Object Tracking,” *IJCV*, vol. 129, pp. 548–578, 2021.
- [5] R. Zhang, P. Isola, A. A. Efros, et al., “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric,” in *CVPR*, 2018.
- [6] M. Heusel, H. Ramsauer, T. Unterthiner, et al., “GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium,” in *NeurIPS*, 2017.
- [7] J. Sturm, N. Engelhard, F. Endres, et al., “A Benchmark for the Evaluation of RGB-D SLAM Systems,” in *IROS*, 2012.
- [8] C. Guo, G. Pleiss, Y. Sun, et al., “On Calibration of Modern Neural Networks,” in *ICML*, 2017.
- [9] C. J. van Rijsbergen, *Information Retrieval*, 2nd ed. London: Butterworths, 1979.
- [10] B. W. Matthews, “Comparison of the Predicted and Observed Secondary Structure of T4 Phage Lysozyme,” *Biochimica et Biophysica Acta (BBA) — Protein Structure*, vol. 405, no. 2, pp. 442–451, 1975.
- [11] D. Chicco and G. Jurman, “The Advantages of the Matthews Correlation Coefficient (MCC) over F1 Score and Accuracy in Binary Classification Evaluation,” *BMC Genomics*, vol. 21, p. 6, 2020.
- [12] J. Cohen, “A coefficient of agreement for nominal scales,” *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960.
- [13] J. Davis and M. Goadrich, “The Relationship Between Precision-Recall and ROC Curves,” in *International Conference on Machine Learning (ICML)*, 2006, pp. 233–240.
- [14] T. Saito and M. Rehmsmeier, “The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets,” *PLoS ONE*, vol. 10, no. 3, e0118432, 2015.
- [15] T. Fawcett, “An Introduction to ROC Analysis,” *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006.
- [16] C. Campos, R. Elvira, J. J. G. Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM,” *IEEE T-RO*, vol. 37, no. 6, pp. 1874–1890, 2021.
- [17] D. Gálvez-López and J. D. Tardós, “Bags of Binary Words for Fast Place Recognition in Image Sequences,” *IEEE Transactions on Robotics*, vol. 28, no. 5, pp. 1188–1197, 2012.
- [18] M. P. Naeini, G. F. Cooper, and M. Hauskrecht, “Obtaining Well Calibrated Probabilities Using Bayesian Binning,” in *AAAI Conference on Artificial Intelligence*, 2015.
- [19] G. W. Brier, “Verification of Forecasts Expressed in Terms of Probability,” *Monthly Weather Review*, vol. 78, no. 1, pp. 1–3, 1950.

- [20] A. H. Murphy, "A New Vector Partition of the Probability Score," *Journal of Applied Meteorology*, vol. 12, no. 4, pp. 595–600, 1973.
- [21] J. Deng, W. Dong, R. Socher, et al., "ImageNet: A Large-Scale Hierarchical Image Database," in *CVPR*, 2009.
- [22] T.-Y. Lin, M. Maire, S. Belongie, et al., "Microsoft COCO: Common Objects in Context," in *ECCV*, 2014.
- [23] J. Hosang, R. Benenson, P. Dollár, et al., "What Makes for Effective Detection Proposals?" *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 38, no. 4, pp. 814–830, 2016.
- [24] D. Hoiem, Y. Chodpathumwan, and Q. Dai, "Diagnosing Error in Object Detectors," in *ECCV*, 2012.
- [25] F. Perazzi, J. Pont-Tuset, B. McWilliams, et al., "A Benchmark Dataset and Evaluation Methodology for Video Object Segmentation," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.
- [26] G. Csurka, D. Larlus, and F. Perronnin, "What Is a Good Evaluation Measure for Semantic Segmentation?" In *British Machine Vision Conference (BMVC)*, 2013.
- [27] Z. Zhang and D. Scaramuzza, "A Tutorial on Quantitative Trajectory Evaluation for Visual(-Inertial) Odometry," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018.
- [28] K. Mikolajczyk and C. Schmid, "A Performance Evaluation of Local Descriptors," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 27, no. 10, pp. 1615–1630, 2005.
- [29] M. Dusmanu, I. Rocco, T. Pajdla, et al., "D2-Net: A Trainable CNN for Joint Description and Detection of Local Features," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [30] B. Efron, "Bootstrap Methods: Another Look at the Jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.
- [31] S. Sagawa, P. W. Koh, T. B. Hashimoto, et al., "Distributionally Robust Neural Networks for Group Shifts," in *ICLR*, 2020.
- [32] P. Henderson, R. Islam, P. Bachman, et al., "Deep Reinforcement Learning That Matters," *AAAI*, 2018.
- [33] K. Musgrave, S. Belongie, and S.-N. Lim, "A Metric Learning Reality Check," in *ECCV*, 2020.
- [34] M. Ferrari Dacrema, P. Cremonesi, and D. Jannach, "Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches," in *RecSys*, 2019.